

Proyek 4AI - Modul 6

Dasar Vision & Interface



016 – Maulana Ishak – 17/04/26

Daftar Isi

MODUL 6: Dasar Vision & Interface.....	3
Tujuan Pembelajaran.....	3
Materi Esensial.....	3
Target Milestones.....	9
Materi untuk Praktikum.....	9
6.1 Evolusi Proyek: Dari Data Tekstual ke Intelijen Visual.....	10
6.2 Functional Requirements (FR) – Edisi Collaborative Logbook.....	10
6.3 Arsitektur Sistem: Hardware Stream ke UI Controller.....	11
6.4 Langkah-Langkah Detail Praktikum.....	11
Langkah 1: Setup Hive & Abstraksi Data Offline (Local Persistence).....	12
Langkah 2: Konstruksi VisionController & Lifecycle Management.....	13
Langkah 3: Implementasi VisionPage & Layering (The Stack).....	16
Langkah 4: Konstruksi DamagePainter (Digital Overlay Logic).....	18
6.5 Materi Pengayaan.....	20
Strategi Frame-per-Second (FPS) dan Analisis Latensi.....	20
Rekomendasi Alur Kerja: "Inference-Ready Architecture".....	21
Tugas Praktikum.....	23
Task 1: Tugas Pendahuluan.....	23
Task 2: The Camera Eye (LOTS).....	23
Task 3: Dynamic Interface Overlay (MOTS).....	23
Task 4: The Mock Detector & Lifecycle Safety (HOTS).....	24
Homework Cosmetic & UX Enhancement (30%).....	24
Penilaian.....	26
📝 Kuesioner Self-Assessment.....	26
👉 Peer Assessment (Apresiasi Rekan Sejawat 1).....	27
👉 Peer Assessment (Apresiasi Rekan Sejawat 2).....	28
👉 Peer Assessment (Apresiasi Rekan Sejawat 3).....	29
🧠 Template Lesson Learnt (Refleksi Akhir).....	30
🤖 Log LLM: The Fact Check & Twist (Homework).....	31
🤖 Contoh Cara Bertanya yang Benar (Prompting).....	31
Rubrik Penilaian Dosen Manajer.....	32
📋 Rubrik Penilaian Log LLM (Integritas).....	32
Referensi.....	34

MODUL 6: Dasar Vision & Interface

Selamat! Anda telah berhasil membangun fondasi data yang tangguh dengan arsitektur *Offline-First* dan sistem keamanan *Gatekeeper* pada modul sebelumnya. Kini, saatnya memberikan 'Mata' pada aplikasi Anda.

Di Modul 6 ini, kita akan melangkah lebih jauh dari sekadar pengolahan teks menuju implementasi **Digital Vision**. Kita akan belajar bagaimana mengintegrasikan *hardware* kamera secara *real-time* dan membangun *Advanced UI* menggunakan `CustomPainter` sebagai lapisan informasi digital.

Di sini, kita akan menerapkan prinsip **Separation of Concerns** dengan memisahkan *Hardware Stream* dari *UI Layering*. Anda akan memahami bagaimana logika koordinat pada Pengolahan Citra Digital (PCD) bertransformasi menjadi antarmuka responsif di perangkat *mobile*. Modul ini adalah langkah awal yang kritical sebelum kita menyuntikkan 'Otak AI' (YOLO) untuk mendeteksi kerusakan jalan secara otomatis."

Tujuan Pembelajaran

Setelah menyelesaikan modul ini, diharapkan Anda mampu:

1. **Memahami Arsitektur Vision pada Mobile:** Menjelaskan cara kerja aliran data (*stream*) dari sensor kamera ke dalam *frame buffer* aplikasi Flutter.
2. **Analisis Resolusi vs Performa:** Menganalisis dampak pemilihan resolusi kamera (Low, Medium, High) terhadap beban komputasi perangkat, sebagai landasan sebelum implementasi model AI (Koneksi PCD).
3. **Memahami Konsep Overlay UI:** Menjelaskan perbedaan antara *Logical Pixels* pada UI Flutter dengan *Physical Pixels* pada sensor kamera untuk akurasi penempatan objek.
4. **Implementasi Camera Lifecycle:** Mampu mengelola inisialisasi dan terminasi (*dispose*) sensor kamera untuk mencegah kebocoran memori (*memory leak*).
5. **Konstruksi Advanced UI dengan CustomPainter:** Membangun lapisan grafis kustom (*Digital Overlay*) di atas *Live Camera Preview* tanpa menghambat performa aplikasi.
6. **Manajemen Permission Native:** Mengonfigurasi izin akses perangkat keras pada level OS (Android/iOS) secara tepat.

Materi Esensial

Sebelum kita mulai menulis kode, pahami dahulu dua pilar utama hari ini:

1. Konsep: Layering UI (Kamera vs Overlay)
 - **Konsep:** Menggunakan Hive sebagai database NoSQL berbasis biner yang berjalan secara lokal di memori perangkat.
 - **Logika:** Berbeda dengan JSON atau Shared Preferences yang memproses data sebagai teks, Hive menyimpan data dalam format biner yang jauh lebih cepat dalam proses baca-tulis, sehingga tidak menghambat *UI thread*.
 - **Tujuan:** Mahasiswa mampu menyediakan penyimpanan data yang tetap tersedia meskipun perangkat tidak memiliki akses internet.

Dalam membangun aplikasi berbasis *Computer Vision*, antarmuka (UI) tidak lagi bersifat statis. Kita menggunakan konsep **Stacking Layer**, di mana layar dibagi menjadi dua lapisan utama yang bekerja secara independen namun tersinkronisasi.

a. Arsitektur Layering

- **Background Layer (Hardware Stream):** Lapisan terbawah yang menjalankan `CameraPreview`. Ini adalah data mentah (*raw data*) dari sensor kamera yang diproses oleh GPU untuk ditampilkan secara *real-time*.
- **Foreground Layer (Digital Overlay):** Lapisan atas menggunakan `CustomPaint`. Di sinilah kita menggambar *bounding box*, teks deteksi, atau *crosshair*. Lapisan ini harus transparan agar kamera di bawahnya tetap terlihat.

b. Implementasi SOLID

Agar kode tidak berantakan (spaghetti code), kita memisahkan logika kamera dari logika menggambar. Kita tidak menggambar kotak deteksi di dalam fungsi kamera, melainkan menggunakan **Painter Pattern**.

Library yang Dibutuhkan: Tambahkan pada `pubspec.yaml`:

```
dependencies:  
  camera: ^0.11.0 # Mengakses hardware kamera  
  permission_handler: ^11.0.0 # Mengelola izin akses secara elegan
```

Konfigurasi Native (AndroidManifest.xml): Mahasiswa sering lupa bahwa Flutter memerlukan izin eksplisit dari OS Android. Tambahkan baris berikut di `android/app/src/main/AndroidManifest.xml`:

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">  
  <uses-permission android:name="android.permission.CAMERA" />  
  <uses-permission android:name="android.permission.RECORD_AUDIO" />  
  
  <uses-feature android:name="android.hardware.camera" />  
  <uses-feature android:name="android.hardware.camera.autofocus" />  
  ...  
</manifest>
```

c. Snippet Implementasi Layering (The Stack Logic)

Gunakan *widget Stack* untuk menumpuk kedua layer ini. Perhatikan bagaimana `CustomPaint` diletakkan **setelah** `CameraPreview`.

```
@override  
Widget build(BuildContext context) {  
  return Scaffold(  
    body: Stack(  
      fit: StackFit.expand,  
      children: [  
        // Layer 1: Kamera (Background)  
        _controller.value.isInitialized  
          ? CameraPreview(_controller)  
          : Container(color: Colors.black),  
      ],  
    ),  
  );  
}
```

```

// Layer 2: Overlay (Foreground)
// Kita menggunakan CustomPaint untuk menggambar indikator
deteksi
    Positioned.fill(
      child: CustomPaint(
        painter: DetectionOverlayPainter(
          targetArea: Rect.fromLTWH(50, 200, 300, 300), // Contoh
area deteksi
        ),
      ),
    ),
  ],
),
);
};
}

```

d. Logika Scaling: Logical vs Physical Pixel

Penting bagi mahasiswa Informatika untuk memahami bahwa koordinat hasil deteksi AI (misal: koordinat x,y dari YOLO) biasanya berupa nilai normalisasi (0 sampai 1).

- **Masalah:** Sensor kamera mungkin beresolusi 1280×720, sedangkan layar HP berukuran 390×844 *logical pixels*.
- **Solusi:** Kita harus mengalikan nilai normalisasi AI dengan ukuran `Size` yang didapat dari `CustomPaint` agar kotak deteksi tepat berada di atas objek (misal: lubang jalan).

Jangan membayangkan kamera sebagai foto diam. Bayangkan kamera sebagai aliran air (stream) yang terus mengalir, dan UI Overlay adalah kaca transparan di depan aliran tersebut yang kita lukis menggunakan spidol digital.

e. Lifecycle Management: Mencegah Memory Leak

Kamera adalah sumber daya *hardware* yang berat. Di Flutter, kita harus secara eksplisit memberi tahu OS untuk melepaskan sensor kamera saat halaman ditutup atau saat aplikasi berpindah ke *background*.

- Logika Dispose (Pelepasan Sumber Daya)
Gunakan prinsip **Resource Acquisition Is Initialization (RAII)**. Setiap kali kita menginisialisasi `CameraController`, kita **wajib** memanggil fungsi `dispose()` ketika widget dihapus dari *tree*.

```

@override
void dispose() {
  // 1. Hentikan aliran stream kamera jika sedang berjalan
  _controller?.stopImageStream();

  // 2. Lepaskan hardware kamera agar bisa digunakan aplikasi lain
  _controller?.dispose();

  super.dispose();
}

```

- Penanganan `AppLifecycleState`

Kebocoran memori sering terjadi saat user menekan tombol "Home" atau menerima telepon. Aplikasi masih berjalan di *background*, tapi kamera tetap menyala. Kita perlu menggunakan `WidgetsBindingObserver`.

Logika:

- o **AppPaused:** Jika aplikasi ke *background*, matikan kamera untuk menghemat baterai dan RAM.
- o **AppResumed:** Jika aplikasi dibuka kembali, inisialisasi ulang kamera.

```
class _VisionPageState extends State<VisionPage> with
WidgetsBindingObserver {
  @override
  void didChangeAppLifecycleState(AppLifecycleState state) {
    final CameraController? cameraController = _controller;

    // Jika aplikasi tidak aktif, lepaskan kamera
    if (cameraController == null ||
!cameraController.value.isInitialized) {
      return;
    }

    if (state == AppLifecycleState.inactive) {
      cameraController.dispose();
    } else if (state == AppLifecycleState.resumed) {
      _setupCamera(); // Inisialisasi ulang saat user kembali
    }
  }
}
```

- Penghindaran `setState` yang Berlebihan

Mahasiswa sering membuat kesalahan dengan memanggil `setState` setiap kali ada *frame* kamera baru (bisa 30-60 kali per detik).

- o **Akibat:** UI *rebuild* terus-menerus, CPU panas.
- o **Solusi:** Gunakan `ValueNotifier` atau cukup biarkan `CameraPreview` yang mengelola rendernya sendiri. Jangan masukkan data citra mentah ke dalam state UI jika tidak diperlukan.

Jika memori bocor, saat kalian mendeteksi pothole (lubang jalan) menggunakan YOLO di Modul 7 nanti, aplikasi akan hang tepat saat ingin menyimpan foto kerusakan tersebut. Pengelolaan memori di Modul 6 adalah fondasi agar AI kalian berjalan stabil.

2. Logika: Koordinat Logical Pixel vs Resolution Frame

Dalam membangun sistem *Smart-Patrol*, mahasiswa harus memahami bahwa ada perbedaan mendasar antara apa yang "dilihat" oleh sensor kamera dan apa yang "digambar" oleh Flutter di layar HP.

a. Perbedaan Sistem Koordinat

- **Resolution Frame (PCD World):** * Ini adalah ukuran fisik gambar yang dihasilkan sensor (misal: `ResolutionPreset.medium` biasanya menghasilkan 1280×720).

- o Output YOLO (Modul 7) akan berada dalam skala ini atau dalam nilai normalisasi (0.0 sampai 1.0).
- **Logical Pixels (Flutter World):** * Ini adalah ukuran layar HP yang digunakan untuk menggambar UI (misal: iPhone 13 memiliki lebar *logical* sekitar 390 unit).
 - o Flutter menggunakan *Device Pixel Ratio* (DPR) untuk menangani densitas layar yang berbeda.
- b. Logika Transformasi (The Scaling Math)

Agar kotak deteksi (Bounding Box) dari dataset **RDD-2022** tepat berada di atas lubang jalan, kita harus melakukan pemetaan (*mapping*).

Rumus Sederhana:

Catatan Penting (Koneksi PCD): > Perlu diingat bahwa sensor kamera pada HP seringkali dalam posisi *landscape* secara *native*. Ketika aplikasi digunakan dalam posisi *portrait*, lebar dan tinggi frame seringkali tertukar (X menjadi Y dan sebaliknya). Inilah alasan mengapa `camera` plugin memiliki properti `sensorOrientation`.

$$\text{Posisi di Layar} = \text{Koordinat AI} \times \frac{\text{Ukuran Layar}}{\text{Ukuran Frame Kamera}}$$

- c. Implementasi Kode: Menghitung Rasio

Berikut adalah *logic snippet* yang harus mahasiswa terapkan di dalam `CustomPainter` mereka untuk memastikan responsivitas di berbagai perangkat:

```
// Di dalam class DetectionPainter extends CustomPainter
@override
void paint(Canvas canvas, Size size) {
  // size di sini adalah ukuran 'Logical Pixel' layar yang tersedia

  // Contoh output mentah dari AI (Normalisasi 0.0 - 1.0)
  // Nantinya berasal dari Output YOLO RDD-2022
  double rawX = 0.5;
  double rawY = 0.3;
  double rawWidth = 0.2;
  double rawHeight = 0.1;

  // Transformasi ke Koordinat Layar
  double finalX = rawX * size.width;
  double finalY = rawY * size.height;
  double finalW = rawWidth * size.width;
  double finalH = rawHeight * size.height;

  final paint = Paint()
    ..color = Colors.red
    ..style = PaintingStyle.stroke
    ..strokeWidth = 3.0;

  // Menggambar Bounding Box di layar
  canvas.drawRect(
    Rect.fromLTWH(finalX, finalY, finalW, finalH),
    paint
  );
}
```




- d. Penerapan SOLID: Open/Closed Principle
- Dengan memisahkan logika **Scaling** ke dalam kelas `Painter` yang mandiri, kita menerapkan **Open/Closed Principle**.
- **Open:** Kita bisa mengganti-ganti cara menggambar (dari kotak merah menjadi ikon lubang jalan) di Modul 7 nanti.
 - **Closed:** Kita tidak perlu mengubah kode di `VisionController` atau `CameraPreview` hanya untuk mengubah tampilan visual deteksi.

3. Tujuan: Standarisasi Vision-Base untuk Studi Kasus RDD-2022

Tujuan utama dari Modul 6 bukanlah sekadar menyalakan kamera, melainkan membangun **Standard Operating Procedure (SOP)** digital untuk deteksi objek infrastruktur. Dengan menggunakan dataset RDD-2022 (Road Damage Detection), kita menetapkan standar bagaimana sebuah "temuan" diproses secara visual.

- a. Mengapa RDD-2022? (Koneksi Domain Teknik Sipil)

Dataset RDD-2022 mengklasifikasikan kerusakan jalan menjadi beberapa tipe spesifik (misal: *Longitudinal Crack*, *Transverse Crack*, *Alligator Crack*, dan *Pothole*).

 - o **Tujuan Standarisasi:** Setiap tipe kerusakan memiliki dimensi yang berbeda. Mahasiswa harus menyiapkan *interface* yang mampu membingkai objek-objek tersebut secara presisi, baik dari sudut pandang kamera HP (patroli darat) maupun simulasi drone.
- b. Standar Frame & Aspek Rasio

Untuk memastikan model YOLO (di Modul 7) bekerja optimal, Modul 6 menetapkan standar **Input Uniformity**:

 - o **Fixed Orientation:** Aplikasi dikunci pada posisi *Portrait* (atau *Landscape* sesuai kebutuhan riset) untuk menghindari pergeseran *bounding box* akibat rotasi layar.
 - o **Visual Anchor:** Penggunaan *Crosshair* atau *Bounding Box* statis berfungsi sebagai panduan bagi user (mahasiswa) saat mengambil data lapangan, sehingga objek kerusakan selalu berada dalam "Area Fokus" AI.
- c. Arsitektur "The Smart-Patrol Template"

ita menerapkan **Template Method Pattern** (Prinsip SOLID). Modul 6 menyediakan "Kerangka Kosong" (Template) yang terdiri dari:

 - o **Vision Engine:** Pengelola aliran data kamera.
 - o **UI Canvas:** Tempat menggambar informasi deteksi.
 - o **Data Bridge:** Jalur komunikasi yang nantinya menghubungkan hasil deteksi AI ke database **MongoDB Atlas** yang sudah dibuat di Modul 5.
- d. Penyiapan Dataset untuk On-Device Inference

Pada tahap ini, mahasiswa diarahkan untuk memahami bahwa:

 - o Gambar yang "dilihat" di layar adalah untuk manusia (User Interface).
 - o *Frame* yang dikirim ke AI (di Modul 7 nanti) adalah untuk mesin (Machine Interface).
 - o **Standarisasi:** Kita akan menggunakan format citra yang ringan namun cukup detail agar bisa mendeteksi retakan kecil pada aspal tanpa membuat HP panas berlebih.

Target Milestones

Tabel berikut mendefinisikan kriteria keberhasilan untuk setiap tahapan praktikum di Modul 6. Mahasiswa diharapkan memenuhi kriteria ini untuk mendapatkan nilai maksimal.

TABEL 6.1: Daftar Target Milestones Modul 6 dan Indikator Keberhasilan.

Milestone	Deskripsi	Kriteria Keberhasilan	Bobot
Vision Setup & Permission	Konfigurasi izin hardware dan inisialisasi dasar kamera.	Aplikasi berhasil meminta izin kamera pada Android/iOS. Preview kamera muncul tanpa <i>crash</i> dan memenuhi seluruh area layar (responsif).	25%
Lifecycle & Resource Safety	Implementasi manajemen memori untuk mencegah <i>leak</i> .	Kode mencakup fungsi <code>dispose()</code> yang benar dan penggunaan <code>WidgetsBindingObserver</code> . Kamera berhenti/menyalakan otomatis saat aplikasi ke <i>background</i> .	25%
Static Overlay UI	Implementasi <code>CustomPainter</code> sebagai lapisan informasi.	Muncul lapisan transparan di atas kamera yang menggambar <i>crosshair</i> atau area pemindaian statis menggunakan koordinat <i>logical pixels</i> .	25%
Dynamic Mock Detection	Logika transformasi koordinat (Scaling) untuk simulasi AI.	Berhasil memindahkan kotak deteksi (Bounding Box) ke titik tertentu secara dinamis melalui kode (sebagai simulasi sebelum integrasi YOLO di Modul 7).	25%

Materi untuk Praktikum

Bagian praktikum modul ini merupakan fase **Sensorisasi** dari proyek *Logbook App* yang telah kita kembangkan. Setelah sebelumnya kita berhasil mengelola data teks secara *offline* (Modul 5), kini kita akan mentransformasi aplikasi tersebut menjadi sistem pemantauan aktif yang mampu berinteraksi dengan dunia fisik melalui kamera.

Fokus transformasi kita kali ini adalah membangun infrastruktur antarmuka yang mampu menjembatani dunia nyata dengan kanvas digital secara *real-time*. Anda tidak lagi hanya berurusan dengan *form* statis, melainkan akan belajar bagaimana mengendalikan *hardware* kamera dan memanipulasi kanvas grafis menggunakan `CustomPainter`. Kita akan menerapkan prinsip **Separation of Concerns** (SOLID) dengan memisahkan *Hardware Stream* (aliran video mentah) dari *UI Layering* (lapisan informasi digital). Modul ini adalah fondasi kritikal; sebuah "indra penglihatan" yang wajib dikuasai sebelum kita menyuntikkan 'Otak AI' (Modul 7) untuk mendeteksi kerusakan jalan secara otomatis.

6.1 Evolusi Proyek: Dari Data Tekstual ke Intelijen Visual

Selamat! Aplikasi Logbook Anda kini telah mencapai tingkat maturitas yang tangguh dalam pengelolaan data dan keamanan. Jika pada **Modul 5** Anda berhasil membangun sistem yang *Resilien* (Offline-First) dan *Kolaboratif* (RBAC), maka di **Modul 6** ini kita akan memulai transformasi besar menuju aplikasi **Intelligent Mobile Research**.

Proyek Anda kini berevolusi dari sekadar aplikasi pencatatan berbasis teks menjadi alat riset lapangan yang memiliki "indera penglihatan". Fase ini menandai transisi dari aplikasi *Enterprise* standar menuju sistem **Smart-Patrol** yang mampu menangkap realitas fisik melalui sensor kamera. Setiap aktivitas pemantauan infrastruktur—seperti deteksi kerusakan jalan pada dataset **RDD-2022**—tidak lagi dicatat secara manual, melainkan dideteksi secara presisi melalui integrasi antarmuka digital di atas aliran video *real-time*.

Lebih jauh lagi, Anda akan menerapkan prinsip **Separation of Concerns** pada level perangkat keras. Data citra mentah dari sensor kamera akan dipisahkan secara arsitektural dari lapisan informasi deteksi (*overlay*). Evolusi ini mempersiapkan aplikasi Anda untuk menerima "Otak AI" pada modul berikutnya, di mana aplikasi tidak hanya menyimpan data yang Anda ketik, tetapi mampu mengenali dan memvalidasi objek kerusakan jalan secara mandiri menggunakan model **YOLO**. Anda sedang membangun masa depan aplikasi inspeksi infrastruktur yang rigid, profesional, dan berbasis data visual.

6.2 Functional Requirements (FR) – Edisi Collaborative Logbook

Tabel berikut merinci kebutuhan fungsional yang harus diimplementasikan pada fase awal pengembangan sistem *Vision*. Mahasiswa wajib memastikan setiap kode yang ditulis merujuk pada pemenuhan kode FR di bawah ini:

ID	Nama Fitur	Deskripsi Kebutuhan Teknis	Output/Hasil
FR-V01	Visual Acquisition	Aplikasi mampu mengakses sensor kamera belakang secara eksklusif dan menampilkan <i>stream</i> video tanpa distorsi.	<i>Live Camera Preview</i>
FR-V02	Overlay Sync	Tersedianya lapisan CustomPaint yang menumpuk secara presisi di atas layer kamera menggunakan widget Stack.	<i>UI Layering</i>
FR-V03	Visual Anchoring	Menampilkan indikator statis (<i>crosshair</i>) sebagai panduan pengguna untuk menargetkan objek kerusakan jalan.	<i>Scanning Guide</i>
FR-V04	Coordinate Scaling	Logika transformasi untuk memetakan nilai normalisasi (0.0 - 1.0) ke dalam koordinat <i>logical pixel</i> layar HP.	<i>Accurate Bounding Box</i>
FR-V05	Lifecycle Safety	Mekanisme pelepasan sumber daya kamera saat aplikasi <i>inactive</i> atau <i>background</i> untuk mencegah kebocoran memori.	<i>Memory Resiliency</i>
FR-V06	Native Permission	Penanganan dialog izin akses kamera secara asinkron sebelum fitur <i>Vision</i> diaktifkan.	<i>Permission Handler</i>

6.3 Arsitektur Sistem: Hardware Stream ke UI Controller

Agar struktur kode tetap **SOLID** dan **Rigid**, arsitektur pada modul ini dibagi menjadi tiga komponen utama:

1. **VisionController (Logic):** Mengelola siklus hidup kamera (*initialize*, *dispose*) dan bertindak sebagai *Observable* bagi UI.
2. **VisionPage (View):** Bertanggung jawab atas tata letak (*Layouting*) dan memastikan aspek rasio kamera sesuai dengan layar perangkat.
3. **DamagePainter (Graphic):** Komponen khusus untuk menggambar elemen grafis (seperti kotak deteksi RDD-2022) tanpa membebani performa *rendering* utama.

6.4 Langkah-Langkah Detail Praktikum

Pada tahap inti ini, Anda akan membangun dan memperluas lapisan antarmuka sensor (*Sensor Interface Layer*) yang berfungsi sebagai pusat kendali persepsi visual antara *hardware* kamera dengan logika aplikasi Flutter. Fokus utama kita tidak lagi sekadar menampilkan gambar, melainkan mentransformasi proyek Anda menjadi sistem yang memiliki "indera penglihatan" digital melalui integrasi *camera* plugin dan lapisan informasi kustom menggunakan *CustomPainter*.

Anda akan diarahkan untuk mengintegrasikan mekanisme **Lifecycle-Aware Camera** agar pengelolaan sumber daya perangkat keras berjalan efisien tanpa menyebabkan kebocoran memori (*memory leak*) atau pemborosan baterai. Selain itu, Anda akan mengimplementasikan protokol **Coordinate Scaling** untuk menjamin bahwa setiap interaksi visual di atas layar—seperti penandaan kerusakan jalan pada dataset **RDD-2022**—berjalan secara presisi dan responsif di berbagai resolusi perangkat. Persiapkan diri Anda untuk menyusun infrastruktur aplikasi yang cerdas, tangguh, dan berstandar *Mobile Engineering* profesional.

Urutan Instruksional Detail Praktikum:

1. **Konfigurasi Native Permission & Library Setup** *Instruksi:* Sinkronisasikan izin akses kamera pada level Sistem Operasi (Android/iOS) dan siapkan *environment* asinkron untuk komunikasi *hardware*.
2. **Konstruksi VisionController & Lifecycle Management** *Instruksi:* Bangun kelas pengendali pusat untuk mengatur inisialisasi sensor, manajemen *frame rate*, dan pelepasan sumber daya secara otomatis.
3. **Implementasi VisionPage & Layering (The Stack)** *Instruksi:* Transformasikan antarmuka aplikasi menjadi struktur berlapis (*stacked*) yang memisahkan aliran video mentah dengan lapisan informasi digital.
4. **Konstruksi DamagePainter (Digital Overlay Logic)** *Instruksi:* Bangun logika kanvas grafis untuk menggambar indikator deteksi kerusakan jalan (seperti *pothole* atau *crack*) berdasarkan koordinat *logical pixels*.

Langkah 1: Setup Hive & Abstraksi Data Offline (Local Persistence)

- **Konsep:** Integrasi izin akses perangkat keras pada level *Kernel* Sistem Operasi. Android dan iOS memerlukan deklarasi eksplisit untuk mengaktifkan sensor kamera demi menjaga privasi pengguna.
- **Logika:** Sinkronisasi antara file manifes (*AndroidManifest.xml*) dengan *asynchronous handler* di Flutter. Hal ini memastikan aplikasi memiliki "kontrak" yang sah dengan OS sebelum mencoba melakukan *handshake* dengan sensor.
- **Tujuan:** Menjamin aplikasi memiliki otoritas legal untuk mengakses kamera dan mencegah *crash* saat inisialisasi pada berbagai versi Android.

Instruksi Detail Praktikum (Step-by-Step)

1.1 Instalasi Dependency Camera & Permission Handler

Buka terminal pada proyek Flutter Anda dan tambahkan paket-paket berikut. Kita menggunakan `camera` untuk akses sensor dan `permission_handler` untuk mengelola dialog izin kepada pengguna secara elegan.

```
flutter pub add camera permission_handler
```

1.2 Konfigurasi Izin Akses Android (Native Level)

Buka file `android/app/src/main/AndroidManifest.xml`. Anda harus mendaftarkan kebutuhan perangkat keras kamera di sini. Jika langkah ini dilewati, aplikasi akan langsung ditutup (*force close*) oleh Sistem Operasi saat mencoba mengakses sensor.

Instruksi: Tambahkan baris berikut di luar tag `<application>`, tepat di bawah tag `<manifest>`.

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
  <uses-permission android:name="android.permission.CAMERA" />

  <uses-feature android:name="android.hardware.camera" />
  <uses-feature android:name="android.hardware.camera.autofocus"
    android:required="false" />

  <application ...>
```

1.3 Penyesuaian Minimum SDK Version

Plugin `camera` membutuhkan fitur API yang hanya tersedia pada versi Android yang lebih baru. Buka file `android/app/build.gradle` (perhatikan folder-nya, jangan tertukar dengan folder `android/build.gradle`).

Instruksi: Cari bagian `defaultConfig` dan ubah `minSdkVersion` menjadi **21**.

```
defaultConfig {
  applicationId "com.example.smart_patrol"
  minSdkVersion 21 // WAJIB: Sesuai standar minimal plugin camera
  targetSdkVersion flutter.targetSdkVersion
  versionCode flutter.versionCode.toInteger()
  versionName flutter.versionName
}
```

1.4 Verifikasi Ketersediaan Kamera di Entry Point

Sebelum memulai praktikum lebih jauh, kita harus memastikan bahwa perangkat (atau emulator) memiliki kamera yang terdeteksi. Modifikasi file `lib/main.dart` untuk mendapatkan daftar kamera yang tersedia saat aplikasi pertama kali dijalankan.

```
import 'package:camera/camera.dart';

List<CameraDescription> cameras = []; // Variabel global untuk menyimpan
daftar kamera
```

```

void main() async {
  WidgetsFlutterBinding.ensureInitialized();

  try {
    // Ambil daftar kamera yang tersedia di perangkat
    cameras = await availableCameras();
  } on CameraException catch (e) {
    print('Error: ${e.code}\nError Message: ${e.description}');
  }

  runApp(const MyApp());
}

```

Troubleshooting Lab (Langkah 1)

- **Masalah:** Muncul pesan error `minSdkVersion 16 is too low` saat menjalankan aplikasi.
- **Solusi:** Pastikan Anda sudah mengubah `minSdkVersion` menjadi **21** di file `android/app/build.gradle` dan melakukan `flutter clean` sebelum menjalankan ulang.
- **Masalah:** `availableCameras()` mengembalikan list kosong di Emulator.
- **Solusi:** Pastikan konfigurasi Emulator Anda sudah mengaktifkan fitur Kamera (biasanya diatur ke *VirtualScene* atau *Webcam0* pada pengaturan AVD Manager).

Langkah 2: Konstruksi VisionController & Lifecycle Management

- **Konsep:** Implementasi *Resource Management* berbasis *Lifecycle-Aware*. Kita menggunakan `WidgetsBindingObserver` untuk memantau status aplikasi (Active, Paused, Inactive).
- **Logika:** Menerapkan mekanisme *Auto-Dispose* dan *Re-Initialization*. Kamera akan dilepaskan (`dispose`) saat aplikasi masuk ke *background* untuk menghemat daya dan RAM, serta diinisialisasi ulang secara otomatis saat aplikasi kembali aktif.
- **Tujuan:** Mencegah kebocoran memori (*memory leak*) dan penggunaan baterai yang berlebihan, memastikan aplikasi tetap *rigid* dan stabil untuk penggunaan riset lapangan dalam waktu lama.

Instruksi Detail Praktikum (Step-by-Step)

2.1 Pembuatan Skeleton Controller

Buat file baru `lib/features/vision/vision_controller.dart`. Kita menggunakan `ChangeNotifier` agar UI bisa bereaksi saat kamera siap, dan `WidgetsBindingObserver` untuk "mendengarkan" sinyal dari Sistem Operasi.

```

import 'package:camera/camera.dart';
import 'package:flutter/material.dart';

class VisionController extends ChangeNotifier with WidgetsBindingObserver {
  CameraController? controller;
  bool isInitialized = false;
  String? errorMessage;

  VisionController() {

```

```

    // Mendaftarkan observer agar bisa memantau status aplikasi (Lifecycle)
    WidgetsBinding.instance.addObserver(this);
    initCamera();
  }
}

```

2.2 Implementasi Inisialisasi Kamera

Tambahkan fungsi `initCamera` di dalam kelas tersebut. Di sini kita menentukan target performa (Resolusi) yang optimal untuk pengolahan citra jalan rusak (RDD-2022).

```

Future<void> initCamera() async {
  try {
    final cameras = await availableCameras();
    if (cameras.isEmpty) {
      errorMessage = "No camera detected on device.";
      notifyListeners();
      return;
    }

    // Memilih Kamera Belakang (Index 0)
    controller = CameraController(
      cameras[0],
      ResolutionPreset.medium, // Keseimbangan antara akurasi AI &
performa
      enableAudio: false,      // Kita hanya butuh visual untuk deteksi
jalan
    );

    await controller!.initialize();
    isInitialized = true;
    errorMessage = null;
  } catch (e) {
    errorMessage = "Failed to initialize camera: $e";
  }
  notifyListeners();
}

```

2.3 Manajemen Siklus Hidup (Lifecycle Handling)

Gunakan *override* pada `didChangeAppLifecycleState`. Ini adalah bagian krusial untuk mencegah HP mahasiswa menjadi panas atau *hang* karena kamera terus berjalan saat mereka sedang membuka aplikasi lain.

```

@override
void didChangeAppLifecycleState(AppLifecycleState state) {
  final CameraController? cameraController = controller;

  // Jika controller belum ada atau belum siap, abaikan
  if (cameraController == null || !cameraController.value.isInitialized) {
    return;
  }

  if (state == AppLifecycleState.inactive) {
    // Melepaskan resource kamera saat aplikasi tidak terlihat
  }
}

```



```

        cameraController.dispose();
        isInitialized = false;
        notifyListeners();
    } else if (state == AppLifecycleState.resumed) {
        // Menginisialisasi ulang saat pengguna kembali ke aplikasi
        initCamera();
    }
}

```

2.4 Cleanup (Dispose)

Pastikan semua "kabel" diputus saat halaman Vision ditutup.

```

@override
void dispose() {
    // Menghapus observer agar tidak terjadi memory leak
    WidgetsBinding.instance.removeObserver(this);
    controller?.dispose();
    super.dispose();
}

```

Troubleshooting Lab (Langkah 2)

- **Masalah:** Kamera tidak kunjung `isInitialized` (Loading terus).
- **Solusi:** Periksa apakah ada aplikasi lain (seperti aplikasi Kamera bawaan atau Zoom) yang sedang menggunakan kamera secara eksklusif. Pastikan juga Langkah 1 (Permission) sudah dilakukan dengan benar.
- **Masalah:** Terjadi error `setNativeView` saat aplikasi di-*resume*.
- **Solusi:** Pastikan logika pengecekan `null` pada `didChangeAppLifecycleState` sudah sesuai dengan kode di atas. Jangan memanggil fungsi kamera pada *controller* yang sudah di-*dispose*.

Langkah 3: Implementasi VisionPage & Layering (The Stack)

- **Konsep:** Arsitektur antarmuka berlapis menggunakan pola *Stacking Layer*. Memisahkan data citra mentah dari perangkat keras dengan elemen grafis buatan aplikasi.
- **Logika:** Menggunakan widget `Stack` dan `AspectRatio` untuk menangani perbedaan dimensi antara *Buffer* kamera dengan *Logical Pixels* layar. Hal ini memastikan gambar tidak mengalami distorsi (gepeng) saat ditampilkan.
- **Tujuan:** Membangun fondasi antarmuka yang mampu menampung informasi digital di atas realitas fisik, sebagai persiapan untuk penempatan *Bounding Box* deteksi jalan rusak.

Instruksi Detail Praktikum (Step-by-Step)

3.1 Inisialisasi State & Controller

Buka file `lib/features/vision/vision_view.dart`. Langkah pertama adalah menghubungkan UI dengan `VisionController` yang telah dibuat di Langkah 2 dan memastikan siklus hidupnya terjaga.

```

class VisionView extends StatefulWidget {
    const VisionView({super.key});
}

```

```

@override
State<VisionView> createState() => _VisionViewState();
}

class _VisionViewState extends State<VisionView> {
  // Inisialisasi controller secara lokal untuk halaman ini
  late VisionController _visionController;

  @override
  void initState() {
    super.initState();
    _visionController = VisionController();
  }

  @override
  void dispose() {
    // WAJIB: Memutus akses kamera saat pindah halaman
    _visionController.dispose();
    super.dispose();
  }
}

```

3.2 Membangun Reactive UI dengan ListenableBuilder

Gunakan `ListenableBuilder` agar UI hanya melakukan *rebuild* pada bagian yang diperlukan saat kamera sudah siap (`isInitialized`).

```

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text("Smart-Patrol Vision")),
    body: ListenableBuilder(
      listenable: _visionController,
      builder: (context, child) {
        // Tampilkan loading jika kamera sedang inisialisasi
        if (!_visionController.isInitialized) {
          return const Center(child: CircularProgressIndicator());
        }

        // Lanjut ke struktur Stack di sub-langkah berikutnya
        return _buildVisionStack();
      },
    ),
  );
}

```

3.3 Implementasi Layered Stack (The "Eyes" & "Overlay")

Di sinilah inti dari arsitektur *Vision*. Kita menggunakan `Stack` dengan `fit: StackFit.expand` agar seluruh lapisan memenuhi layar.

```

Widget _buildVisionStack() {
  return Stack(
    fit: StackFit.expand,
    children: [

```

```

// LAYER 1: Hardware Preview
// Menggunakan AspectRatio agar gambar tidak gepeng (Koneksi PCD)
Center(
  child: AspectRatio(
    aspectRatio: _visionController.controller!.value.aspectRatio,
    child: CameraPreview(_visionController.controller!),
  ),
),

// LAYER 2: Digital Overlay (Canvas)
// Layer ini transparan dan berada tepat di atas kamera
const Positioned.fill(
  child: CustomPainter(
    painter: DamagePainter(), // Langkah 4
  ),
),
],
);
}

```

3.4 Navigasi & Entry Point

Agar halaman ini bisa diakses, tambahkan navigasi pada menu utama atau *FloatingActionButton* di *LogView*.

```

// Contoh penempatan di FloatingActionButton LogView
floatingActionButton: FloatingActionButton(
  onPressed: () => Navigator.push(
    context,
    MaterialPageRoute(builder: (context) => const VisionView())
  ),
  child: const Icon(Icons.camera_alt),
),

```

Troubleshooting Lab (Langkah 3)

- **Masalah:** Muncul layar hitam (Black Screen) meskipun sudah pindah halaman.
- **Solusi:** Periksa `errorMessage` pada `VisionController`. Biasanya terjadi karena `permission` belum disetujui atau `minSdkVersion` belum diubah ke 21.
- **Masalah:** Gambar kamera meluber keluar layar (Overflow).
- **Solusi:** Pastikan `Stack` dibungkus oleh `Scaffold` dan gunakan `fit: StackFit.expand` untuk membatasi area render.

Langkah 4: Konstruksi DamagePainter (Digital Overlay Logic)

- **Konsep:** Visualisasi data spasial menggunakan koordinat sistem *Canvas*. Kita memindahkan logika penggambaran dari *Widget Tree* utama ke dalam kelas `CustomPainter`.
- **Logika:** Mengimplementasikan fungsi `drawRect` dan `TextPainter` untuk menciptakan indikator visual statis. Lapisan ini nantinya akan menerima *stream* koordinat dari model YOLO untuk menandai area kerusakan jalan.

- **Tujuan:** Memberikan panduan visual kepada pengguna (mahasiswa) saat melakukan patroli jalan dan menstandarisasi cara aplikasi memvisualisasikan "temuan" berdasarkan dataset **RDD-2022**.

Instruksi Detail Praktikum (Step-by-Step)

4.1 Definisi Class & Paint Configuration

Buat file baru `lib/features/vision/damagePainter.dart`. Langkah awal adalah mendefinisikan objek `Paint` yang akan menentukan karakteristik visual (warna, ketebalan garis) dari kotak deteksi.

```
import 'package:flutter/material.dart';

class DamagePainter extends CustomPainter {
  @override
  void paint(Canvas canvas, Size size) {
    // 1. Konfigurasi "Kuas" Digital
    final paint = Paint()
      ..color = Colors.redAccent
      ..strokeWidth = 3.0
      ..style = PaintingStyle.stroke; // Garis pinggir saja, bukan blok
    warna

    // Lanjut ke logika penggambaran di sub-langkah berikutnya...
  }
}
```

4.2 Logika Penempatan Bounding Box (Anchoring)

Mahasiswa belajar menempatkan kotak deteksi simulasi. Pada tahap ini, kita menggunakan persentase dari ukuran layar (`Size`) agar kotak tetap proporsional di berbagai perangkat (Prinsip *Responsive UI*).

```
// 2. Menghitung Dimensi Kotak (Simulasi Pothole RDD-2022)
// Membuat kotak di tengah layar seluas 50% dari lebar layar
double boxSize = size.width * 0.5;
double left = (size.width - boxSize) / 2;
double top = (size.height - boxSize) / 2;

final rect = Rect.fromLTWH(left, top, boxSize, boxSize);

// 3. Menggambar Kotak ke Kanvas
canvas.drawRect(rect, paint);
```

4.3 Rendering Label Intelijen (Text Painting)

Berbeda dengan widget `Text`, menggambar teks di dalam `CustomPainter` memerlukan `TextPainter`. Mahasiswa harus memahami alur *Layouting* teks sebelum bisa digambar (*painted*) ke koordinat tertentu.

```
// 4. Konstruksi Label Tipe Kerusakan
const textStyle = TextStyle(
  color: Colors.white,
  fontSize: 14,
  fontWeight: FontWeight.bold,
```

```

        backgroundColor: Colors.redAccent,
    );

    final textSpan = TextSpan(
        text: " [D40] POTHOLE - 92% ", // Kode D40 merujuk pada RDD-2022
        style: textStyle,
    );

    final textPainter = TextPainter(
        text: textSpan,
        textDirection: TextDirection.ltr,
    );

    // 5. Proses Layouting & Rendering Teks
    textPainter.layout();
    // Gambar teks tepat di atas garis kotak (offset -25 pixel)
    textPainter.paint(canvas, Offset(left, top - 25));

```

4.4 Optimasi Repaint Logic

Mahasiswa harus memahami kapan kanvas perlu menggambar ulang. Untuk Modul 6 (statis), kita mengembalikan `false`, namun di Modul 7 (dinamis) kita akan mengubahnya menjadi `true`.

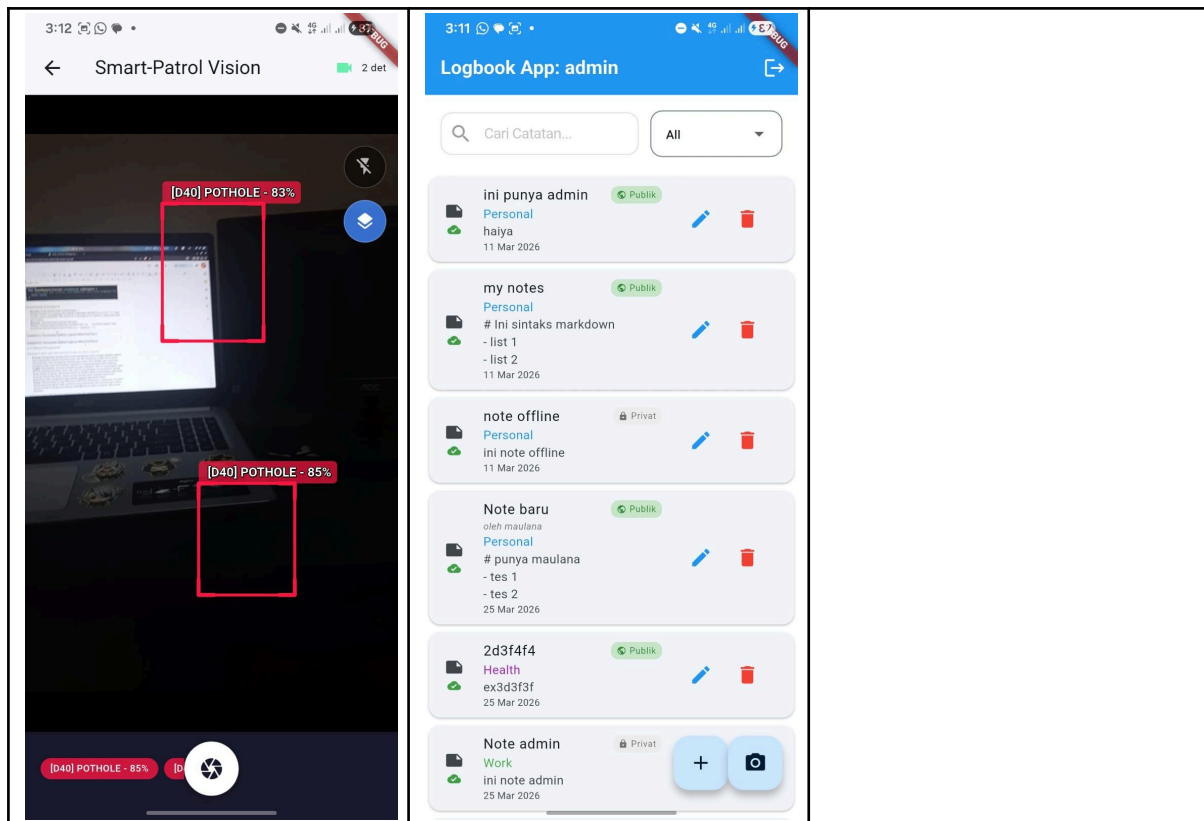
```

@override
bool shouldRepaint(covariant CustomPainter oldDelegate) {
    // Untuk tahap awal (statis), kita kembalikan false untuk menghemat CPU.
    return false;
}

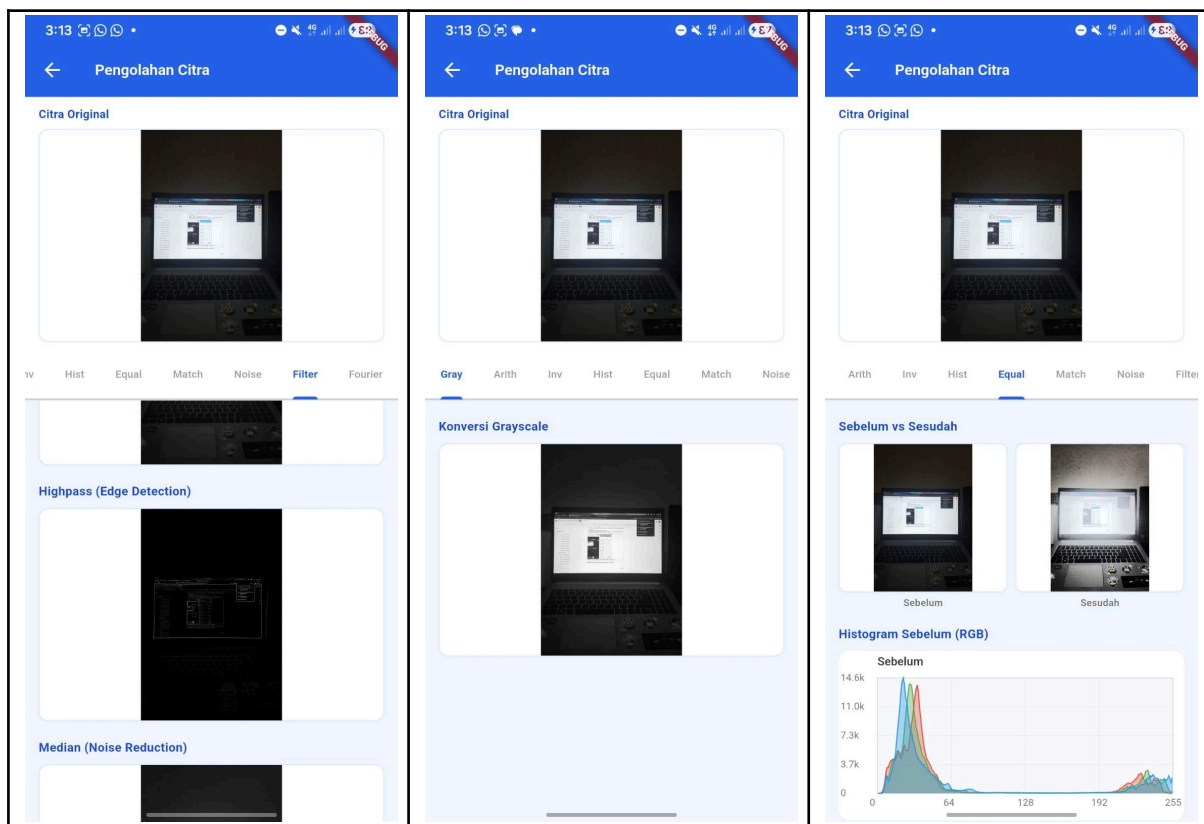
```

Troubleshooting Lab (Langkah 4)

- **Masalah:** Kotak deteksi tidak muncul di layar.
- **Solusi:** Pastikan `CustomPaint` di Langkah 3 dibungkus dengan `Positioned.fill` atau memiliki ukuran yang jelas. Jika ukuran `Size` di fungsi `paint` adalah 0, maka kotak tidak akan terlihat.
- **Masalah:** Teks label terpotong oleh tepi layar.
- **Solusi:** Gunakan logika pengecekan koordinat; jika `top - 25` bernilai negatif, maka pindahkan label ke bagian bawah kotak (`top + boxSize + 5`).



GAMBAR 5.1: Penampakan aplikasi Logbook Offline First Part 1.



GAMBAR 5.2: Penampakan aplikasi Logbook Offline First Part 2.

6.5 Materi Pengayaan

Strategi Frame-per-Second (FPS) dan Analisis Latensi

- **Konsep:** Mengadopsi strategi *Performance Budgeting* untuk menjaga stabilitas aplikasi saat menjalankan beban komputasi berat. Alih-alih memproses setiap *frame* secara membabi buta, kita menetapkan ambang batas waktu (*time budget*) agar antarmuka pengguna tetap responsif meskipun algoritma AI sedang bekerja di latar belakang.
- **Logika:** Memisahkan alur kerja menjadi dua jalur (*Pipeline*): Jalur UI yang berjalan pada 60 FPS untuk kenyamanan visual, dan Jalur AI yang berjalan secara asinkron dengan teknik *Frame Dropping*. Jika proses *inference* (deteksi objek) membutuhkan waktu lebih lama dari durasi satu *frame*, sistem secara otomatis akan melewati (*skip*) *frame* berikutnya untuk menghindari penumpukan antrean data (*buffer bloat*).
- **Tujuan:** Menanamkan prinsip *Efficiency & Resource Awareness*; mahasiswa menyadari bahwa pada perangkat *mobile*, performa yang optimal lahir dari keseimbangan antara akurasi model AI dengan kemampuan perangkat keras dalam mengolah data secara *real-time*.

Rekomendasi Alur Kerja: "Inference-Ready Architecture"

Untuk mempersiapkan Modul 7, mahasiswa disarankan mulai memikirkan bagaimana data citra dari **Langkah 3** akan dikirimkan ke model YOLO tanpa menghentikan *preview* kamera.

1. Dekomposisi Masalah: Pemisahan Logika Deteksi dan UI
 - **Konsep:** Menerapkan *Single Responsibility Principle* (SRP). Jangan mencampur logika matematika deteksi di dalam fungsi `build` atau `Painter`.
 - **Logika:** Buatlah sebuah *Data Transfer Object* (DTO) sederhana bernama `DetectionResult`. `VisionController` bertugas menghasilkan objek ini, sementara `DamagePainter` hanya bertugas menggambarkannya.
 - **Skenario:** Jika suatu saat Anda mengganti model YOLO dengan model lain, Anda cukup mengubah pengisi data di `VisionController` tanpa perlu menyentuh kode UI atau `Painter`.

```
// 1. Definisikan Objek Data (DTO)
class DetectionResult {
    final Rect box;        // Koordinat kotak
    final String label;    // Tipe kerusakan (D40, D20, dll)
    final double score;    // Persentase keyakinan AI

    DetectionResult({required this.box, required this.label,
required this.score});
}

// 2. Di dalam Painter, cukup terima list hasil deteksi
class DamagePainter extends CustomPainter {
    final List<DetectionResult> results;
    DamagePainter(this.results);

    @override
    void paint(Canvas canvas, Size size) {
        for (var res in results) {
            // Logika gambar kotak di sini menggunakan res.box
        }
    }
}
```



```
}
}
}
```

2. Asynchronous Processing: Menjaga Kelancaran UI (Isolates)

- **Konsep:** *Concurrency* untuk beban kerja berat. Di Dart, UI berjalan di *Main Isolate*. Jika proses PCD (seperti *image resizing*) dilakukan di sini, UI akan "macet".
- **Logika:** Gunakan fungsi `compute` (Isolate sederhana) untuk memindahkan beban pemrosesan citra ke *thread* yang berbeda.
- **Skenario:** Bayangkan aplikasi harus mengubah ukuran gambar dari kamera setiap 100ms. Dengan Isolate, proses ini dilakukan di "dapur" yang berbeda, sehingga animasi di layar tetap berjalan halus (60 FPS).

```
// Fungsi yang berjalan di Isolate terpisah (Statik/Global)
Future<List<DetectionResult>> heavyInferenceTask(CameraImage image)
async {
  // Simulasi beban PCD yang berat
  // 1. Image Pre-processing
  // 2. Run Model Inference
  return []; // Kembalikan hasil ke Main Isolate
}

// Di VisionController
void processFrame(CameraImage image) async {
  if (isProcessing) return; // Skip frame jika AI masih sibuk

  isProcessing = true;
  // Menjalankan tugas berat tanpa membebani UI
  final results = await compute(heavyInferenceTask, image);

  _currentResults = results;
  notifyListeners();
  isProcessing = false;
}
```

3. Scaling Factor: Sinkronisasi Ruang Koordinat

- **Konsep:** *Spatial Mapping*. Mengubah koordinat dari ruang sensor (misal: 640x640) ke ruang layar (misal: 1080x2400).
- **Logika:** Gunakan nilai normalisasi (0.0 sampai 1.0). AI akan memberikan output: "Ada lubang di X=0.5". Kode Anda harus menghitung: "0.5 dari lebar layar HP ini adalah 200 piksel".
- **Skenario:** Mahasiswa menggunakan berbagai jenis HP dengan ukuran layar berbeda. Tanpa *Scaling Factor* yang rigid, kotak deteksi yang muncul di HP layar lebar akan meleset jika dibandingkan dengan HP layar kecil.

```
void paint(Canvas canvas, Size size) {
  // size di sini adalah ukuran LAYAR HP yang aktif (Logical Pixels)

  // Contoh output AI dalam nilai normalisasi (0.0 - 1.0)
  double aiX = 0.2;
```

```

double aiY = 0.5;

// Rumus Scaling yang Rigid
double finalX = aiX * size.width;
double finalY = aiY * size.height;

canvas.drawCircle(Offset(finalX, finalY), 10, Paint()..color =
Colors.yellow);
}

```

Tugas Praktikum

Untuk menguji pemahaman Anda mengenai arsitektur *Vision Interface* dan *Lifecycle Management*, selesaikan tantangan berikut. Aturan main: 70% fungsionalitas inti (Kamera & Painter) harus selesai di Lab, dan 30% sisa (Animasi & UX Enhancement) menjadi *homework* portofolio Anda.

Task 1: Tugas Pendahuluan

Dikerjakan secara mandiri sebelum sesi praktikum dimulai.

1. Apa perbedaan mendasar antara *Logical Pixels* di Flutter dengan *Physical Pixels* pada sensor kamera? Mengapa pemahaman ini penting untuk deteksi objek?
2. Mengapa penggunaan `WidgetsBindingObserver` sangat kritikal saat kita membangun aplikasi yang mengakses *hardware* kamera?
3. Berdasarkan dataset **RDD-2022**, sebutkan 4 tipe kerusakan jalan utama dan kode labelnya (misal: D00, D10, dst).

Jawab:

1. Logical Pixels di Flutter merupakan satuan abstrak yang ukurannya sama di semua device, jadi kalau kita tulis `width: 100`, tampilannya akan kelihatan sama besar di HP apapun. Sedangkan Physical Pixels merupakan piksel asli dari sensor kamera, jumlahnya tergantung resolusi hardware (misal 1920×1080). Ini penting untuk deteksi objek karena model AI mendeteksi di koordinat physical pixels kamera, tapi kita gambar bounding box-nya di layar Flutter yang pakai logical pixels. Kalau tidak di-convert pakai scaling factor (device pixel ratio), kotak deteksinya akan geser atau salah posisi. Maka dari itu, gunakan koordinat normalisasi (0.0–1.0) agar bisa di-mapping ke ukuran layar apapun.
2. `WidgetsBindingObserver` itu kritikal karena dia mendeteksi kapan app di-minimize, di-lock, atau di-switch ke app lain (`AppLifecycleState`). Kalau kita tidak lepas resource kamera saat app ke background, akibatnya:
 - Kamera tetap nyala → baterai boros, HP panas
 - Resource tidak dilepas → app lain tidak bisa akses kamera
 - Crash saat balik ke app karena kamera dalam state tidak konsisten

Jadi di `didChangeAppLifecycleState`, kita wajib dispose kamera saat `paused/inactive` dan init ulang saat `resumed`.

3. Tipe Kerusakan Jalan (RDD-2022)

Kode	Tipe Kerusakan	Deskripsi
D00	Longitudinal Crack	Retak memanjang sejajar arah jalan
D10	Transverse Crack	Retak melintang tegak lurus arah jalan
D20	Alligator Crack	Retak buaya/jaring, area luas saling terhubung
D40	Pothole	Lubang pada permukaan jalan

Task 2: The Camera Eye (LOTS)

Fokus: Mengimplementasikan inisialisasi kamera dan penanganan izin (*permission*).

Spesifikasi Tugas:

1. **Native Setup:** Konfigurasi `AndroidManifest.xml` dan `minSdkVersion` sesuai standar plugin kamera.
2. **Controller Logic:** Bangun `VisionController` yang mampu melakukan inisialisasi kamera belakang dengan resolusi `medium`.
3. **Live Preview:** Tampilkan aliran video kamera pada halaman `VisionView` tanpa distorsi aspek rasio.

Kriteria Selesai di Lab (70%):

- [X] Berhasil melewati dialog *permission* kamera saat aplikasi pertama kali dijalankan.
- [X] *Live Preview* muncul di layar HP/Emulator tanpa membuat aplikasi *crash*.
- [X] Menggunakan `AspectRatio` untuk memastikan gambar tidak terlihat gepeng/lonjong.

Task 3: Dynamic Interface Overlay (MOTS)

Fokus: Membangun lapisan informasi digital menggunakan `CustomPainter`.

Spesifikasi Tugas:

1. **Static Anchor:** Implementasikan `CustomPaint` yang menggambar indikator *crosshair* atau area pemindaian di tengah layar.
2. **Vision Label:** Tambahkan teks informasi "Searching for Road Damage..." yang menempel pada indikator tersebut menggunakan `TextPainter`.
3. **Rigid Positioning:** Pastikan posisi *overlay* tetap presisi di tengah layar terlepas dari perbedaan ukuran layar perangkat.

Kriteria Selesai di Lab (70%):

- [X] Lapisan grafis (Painter) muncul dengan jelas di atas aliran video kamera.
- [X] Teks label terbaca dengan kontras yang baik (menggunakan *background color* pada teks).
- [X] Tidak terjadi penurunan *frame rate* (FPS) yang signifikan saat *overlay* diaktifkan.

Task 4: The Mock Detector & Lifecycle Safety (HOTS)

Fokus: Simulasi deteksi objek (RDD-2022) dan ketangguhan manajemen memori.

Spesifikasi Tugas:

1. **Mock Detection Logic:** Buatlah fungsi yang secara otomatis memindahkan kotak deteksi ke titik acak di layar setiap 3 detik (sebagai simulasi output YOLO).
2. **Scaling Calibration:** Pastikan kotak deteksi memiliki ukuran yang proporsional terhadap lebar layar (menggunakan variabel `Size`).
3. **Resource Guard:** Implementasikan logika *Auto-Dispose*; kamera harus mati saat menekan tombol 'Back' atau saat aplikasi masuk ke *background*.

Kriteria Selesai di Lab (70%):

- [X] Kotak deteksi bergerak secara dinamis sesuai logika simulasi tanpa menyebabkan *flicker*.
- [X] Kamera berhasil berhenti (*dispose*) secara otomatis saat diperiksa melalui *memory profiler* atau indikator privasi OS.
- [X] Menunjukkan bukti *Log LLM* yang mencantumkan analisis terhadap *Scaling Factor* koordinat.

Tugas Pengayaan (Bridging to Modul 7): The Latency Test (HOTS) Cobalah ubah

`ResolutionPreset` ke `max` atau `ultraHigh`. Amati suhu perangkat dan kelancaran UI.

Dokumentasikan dalam *Lesson Learnt*: "Mengapa resolusi tinggi justru merugikan untuk aplikasi patroli jalan raya?"

Homework Cosmetic & UX Enhancement (30%)

Setelah logika inti (*Core Logic*) berhasil Anda selesaikan di laboratorium, gunakan waktu mandiri Anda di rumah untuk meningkatkan standar kualitas aplikasi (UX) melalui tiga fitur berikut:

1. Smart Vision Toggle & Flashlight (Hardware Control Practice)
Tambahkan kontrol manual untuk memberikan fleksibilitas kepada pengguna saat melakukan inspeksi jalan di berbagai kondisi pencahayaan.
 - o **Tugas:**
 - Implementasikan tombol *Toggle* untuk menyalakan dan mematikan lampu *flash* (*Torch*) pada perangkat melalui `VisionController`.
 - Tambahkan tombol *Switch* untuk mengaktifkan/menonaktifkan lapisan *Overlay Painter* secara *real-time*.
 - o **Tujuan:** Melatih kemampuan mahasiswa dalam mengendalikan fitur spesifik *hardware* dan mengelola status *UI Layering* secara dinamis.
2. Informative Vision State (User Feedback)
Pastikan pengguna memahami apa yang sedang terjadi di balik layar saat aplikasi sedang melakukan proses inisialisasi atau pemindaian.
 - o **Tugas:**
 - Gunakan animasi *Lottie* atau `CircularProgressIndicator` yang dikustomisasi dengan teks instruksional saat kamera sedang *loading* (misal: "Menghubungkan ke Sensor Visual...").

- Tampilkan pesan "No Camera Access" yang informatif disertai tombol "Open Settings" jika izin kamera ditolak oleh pengguna.
- o **Tujuan:** Memberikan umpan balik visual yang ramah dan solutif kepada pengguna sesuai standar aplikasi *enterprise* modern.
- 3. Detection Style & Color Branding (Visual Communication)

Tingkatkan aspek keterbacaan deteksi agar informasi kerusakan jalan (RDD-2022) dapat dibedakan dengan mudah secara visual.

 - o **Tugas:**
 - Implementasikan skema warna dinamis pada `DamagePainter`:
Gunakan warna **Merah** untuk tipe kerusakan berat (misal: *Pothole D40*) dan warna **Kuning** untuk kerusakan ringan (misal: *Longitudinal Crack D00*).
 - Tambahkan efek *Stroke* luar atau bayangan (*Shadow*) pada teks label agar tetap terbaca jelas meskipun latar belakang jalan memiliki warna yang serupa dengan teks.
 - o **Tujuan:** Mempermudah pengguna (petugas lapangan) dalam melakukan klasifikasi cepat berdasarkan hierarki visual warna saat melakukan patroli.

Penilaian



Kuesioner Self-Assessment

Jawablah dengan jujur sesuai dengan apa yang kalian rasakan setelah menyelesaikan Modul 6. Berikan tanda centang (✓) pada kolom angka yang paling sesuai:

Keterangan Skala: 1: Sangat Tidak Setuju (STS) | 2: Tidak Setuju (TS) | 3: Netral (N) | 4: Setuju (S) | 5: Sangat Setuju (SS)

No	Pernyataan Refleksi	1	2	3	4	5
1	Saya memahami peran <i>Lifecycle Management</i> dalam mencegah <i>memory leak</i> pada penggunaan hardware kamera.			✓		
2	Saya mampu membedakan sistem koordinat <i>Logical Pixels</i> (UI) dengan <i>Frame Resolution</i> (Hardware).			✓		
3	Saya memahami cara kerja widget <i>Stack</i> dalam memisahkan lapisan aliran video mentah dengan lapisan grafis digital.			✓		
4	Saya mampu mengimplementasikan <i>CustomPainter</i> untuk menggambar informasi deteksi objek secara presisi.			✓		
5	Saya memahami pentingnya pemilihan <i>ResolutionPreset</i> terhadap keseimbangan antara akurasi AI dan performa perangkat.			✓		
6	Saya memahami alur normalisasi koordinat yang akan digunakan untuk integrasi model YOLO RDD-2022 di modul berikutnya.			✓		

Apa bagian yang paling menantang atau membingungkan bagi kalian di modul ini?

Mengimplementasikan fourier transform

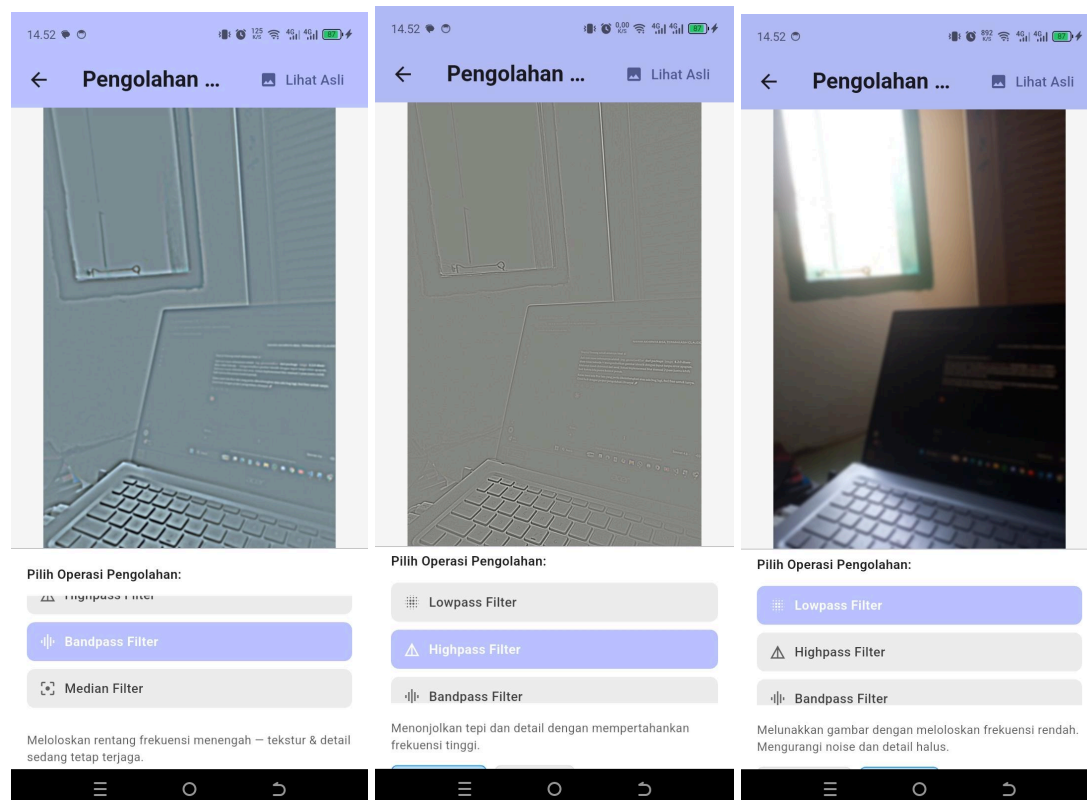
👉 Peer Assessment (Apresiasi Rekan Sejawat 1)

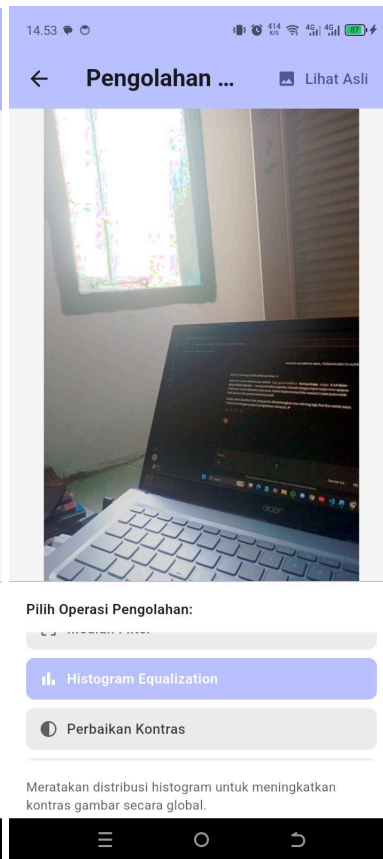
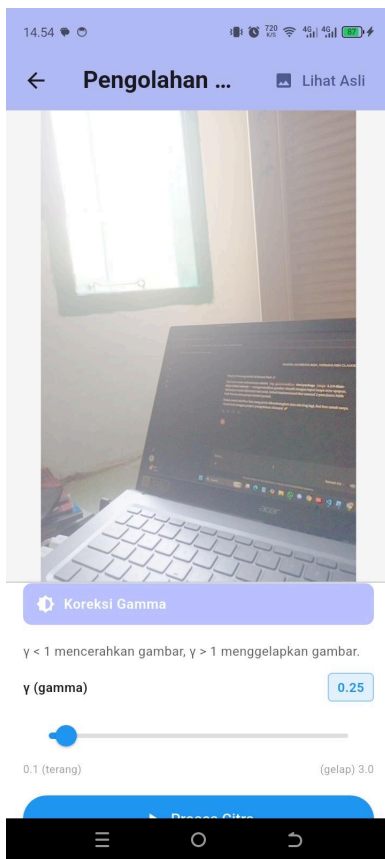
Setelah selesai, tukarkan HP kalian dengan teman sebangku. Cobalah aplikasi mereka dan berikan penilaian sejujur mungkin. Ingat, tujuannya bukan menjatuhkan, tapi saling memperbaiki!

Nama Penilai: Maulana Ishak **Nama Pemilik Aplikasi:** Dwika

No	Kriteria Pengalaman Pengguna (UX)	Skor (1-5)	Catatan "Hal Keren/Saran"
1	Kestabilan Kamera: Preview kamera tidak pecah/lag dan tidak terjadi <i>crash</i> saat berpindah-pindah halaman.	5	
2	Akurasi Overlay: Kotak deteksi (Painter) berada tepat di posisi yang diharapkan dan responsif terhadap perubahan layar.	5	
3	Ergonomi UI: Letak label informasi tidak menghalangi objek utama di kamera dan teks mudah dibaca.	5	
4	Efisiensi Memori: Perangkat tidak mengalami panas berlebih atau perlambatan signifikan saat fitur vision aktif.	5	

Print Screen Aplikasi Teman Anda:





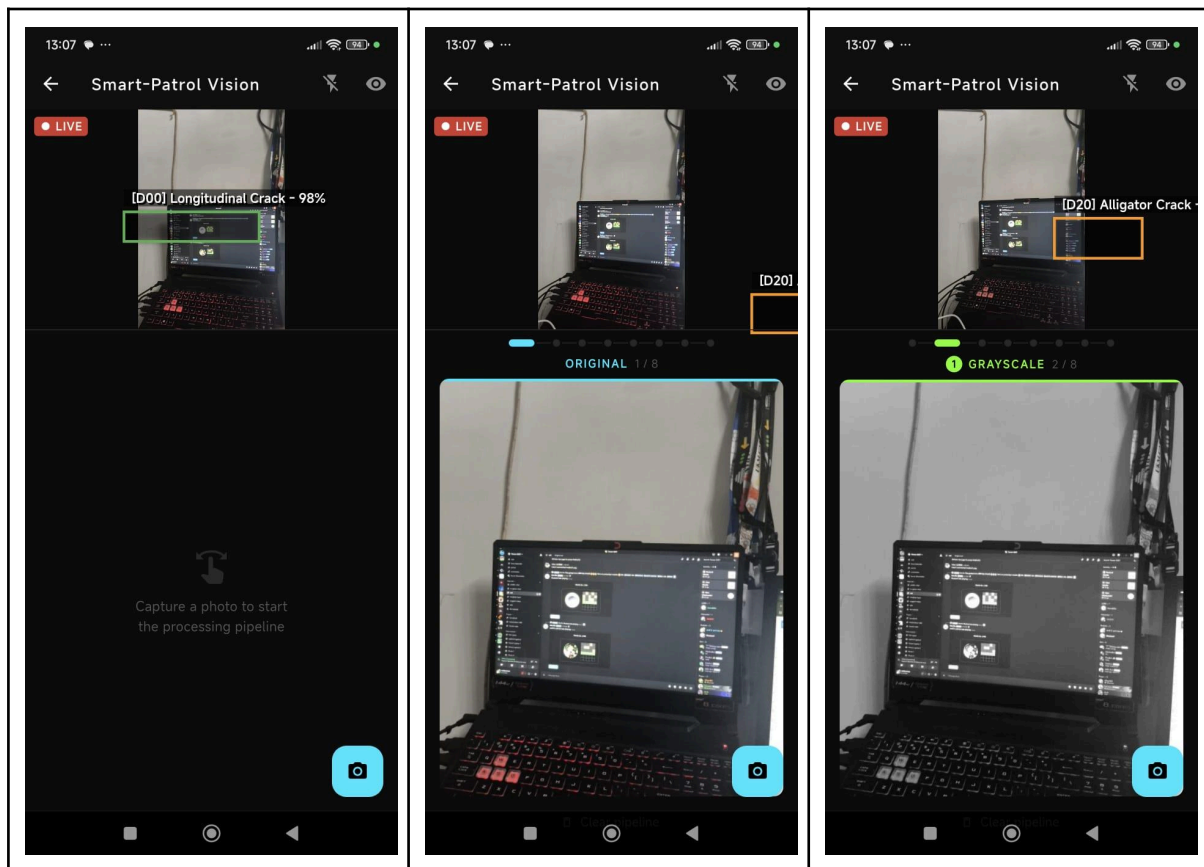
👋 Peer Assessment (Apresiasi Rekan Sejawat 2)

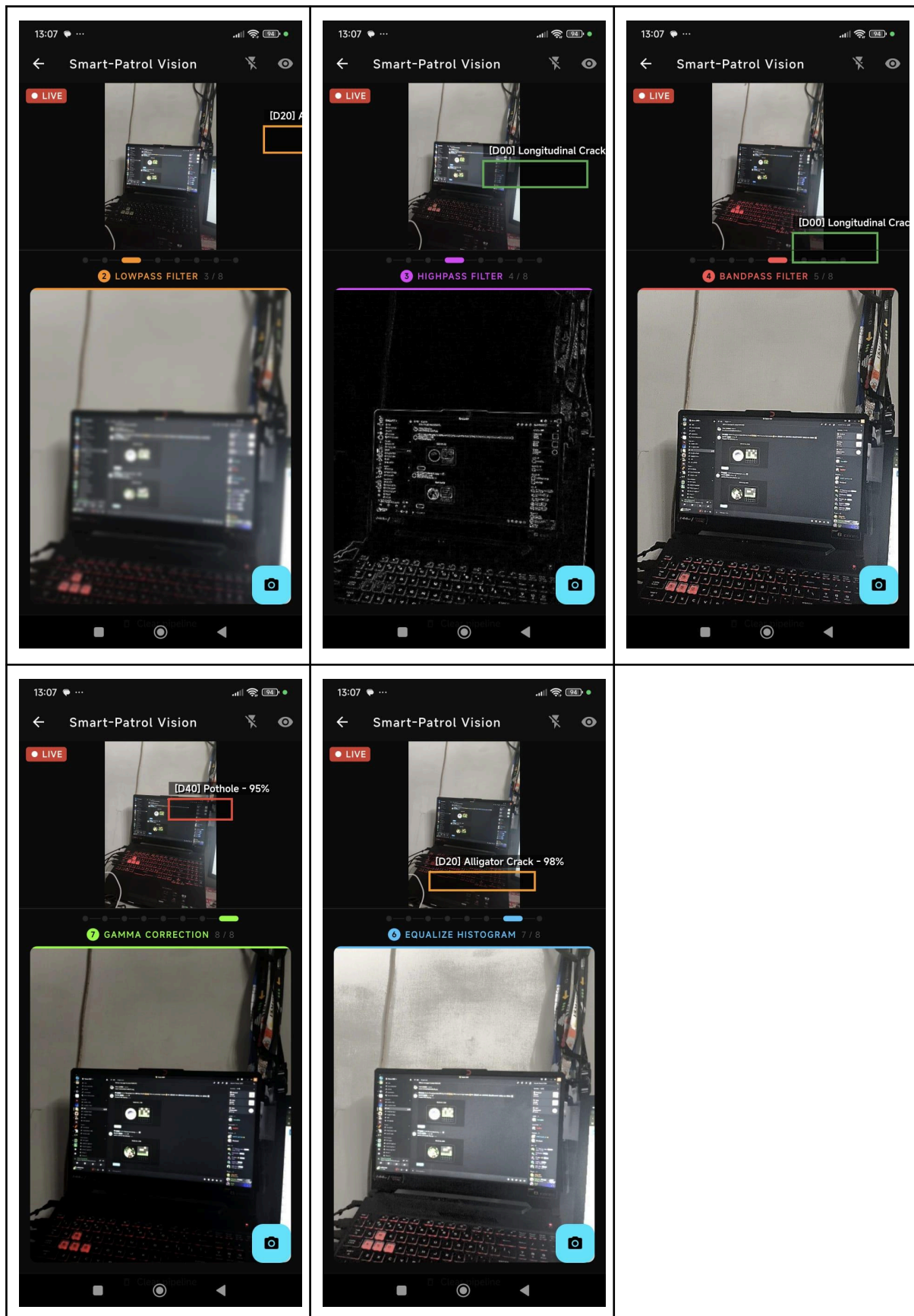
Setelah selesai, tukarkan HP kalian dengan teman sebangku. Cobalah aplikasi mereka dan berikan penilaian sejujur mungkin. Ingat, tujuannya bukan menjatuhkan, tapi saling memperbaiki!

Nama Penilai: Maulana Ishak **Nama Pemilik Aplikasi:** Andi Putra Wijaya

No	Kriteria Pengalaman Pengguna (UX)	Skor (1-5)	Catatan "Hal Keren/Saran"
1	Kestabilan Kamera: Preview kamera tidak pecah/lag dan tidak terjadi <i>crash</i> saat berpindah-pindah halaman.	5	
2	Akurasi Overlay: Kotak deteksi (Painter) berada tepat di posisi yang diharapkan dan responsif terhadap perubahan layar.	5	
3	Ergonomi UI: Letak label informasi tidak menghalangi objek utama di kamera dan teks mudah dibaca.	5	
4	Efisiensi Memori: Perangkat tidak mengalami panas berlebih atau perlambatan signifikan saat fitur vision aktif.	5	

Print Screen Aplikasi Teman Anda:





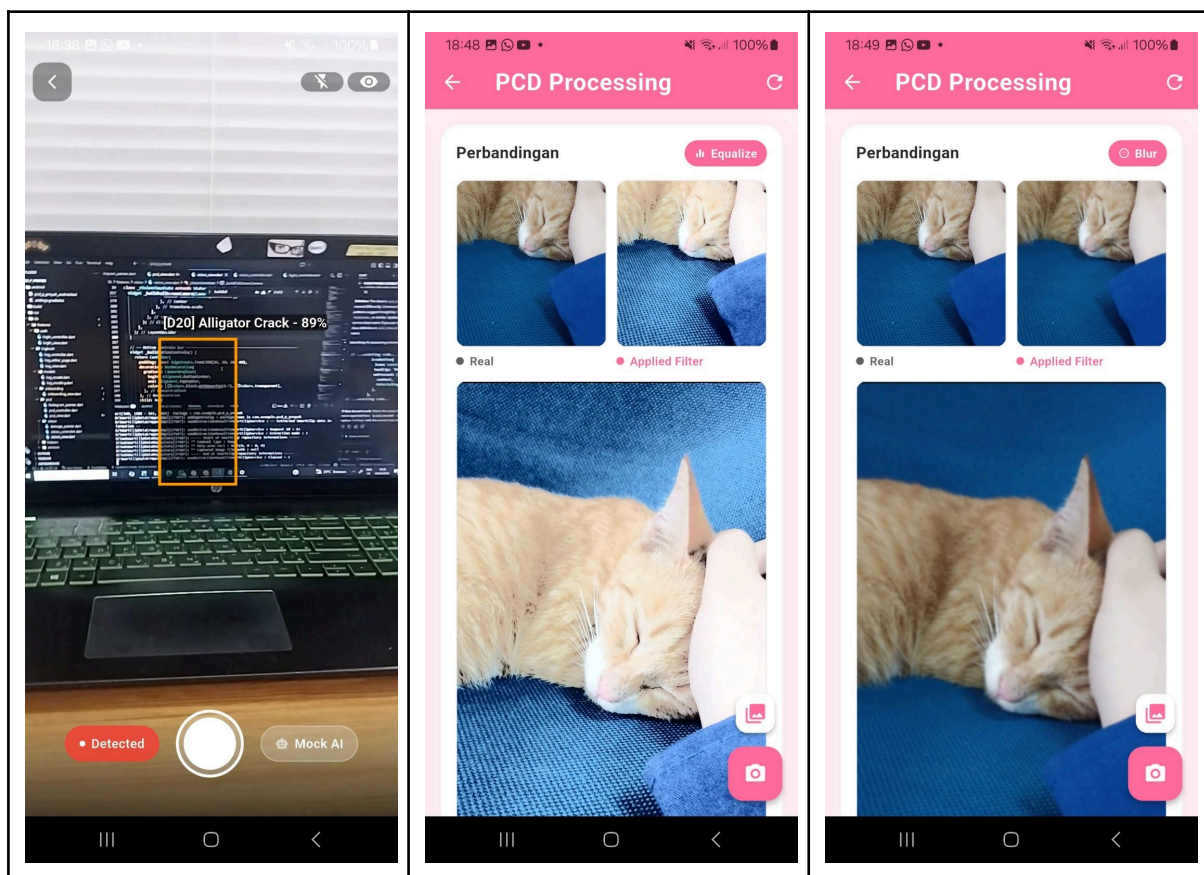
👉 Peer Assessment (Apresiasi Rekan Sejawat 3)

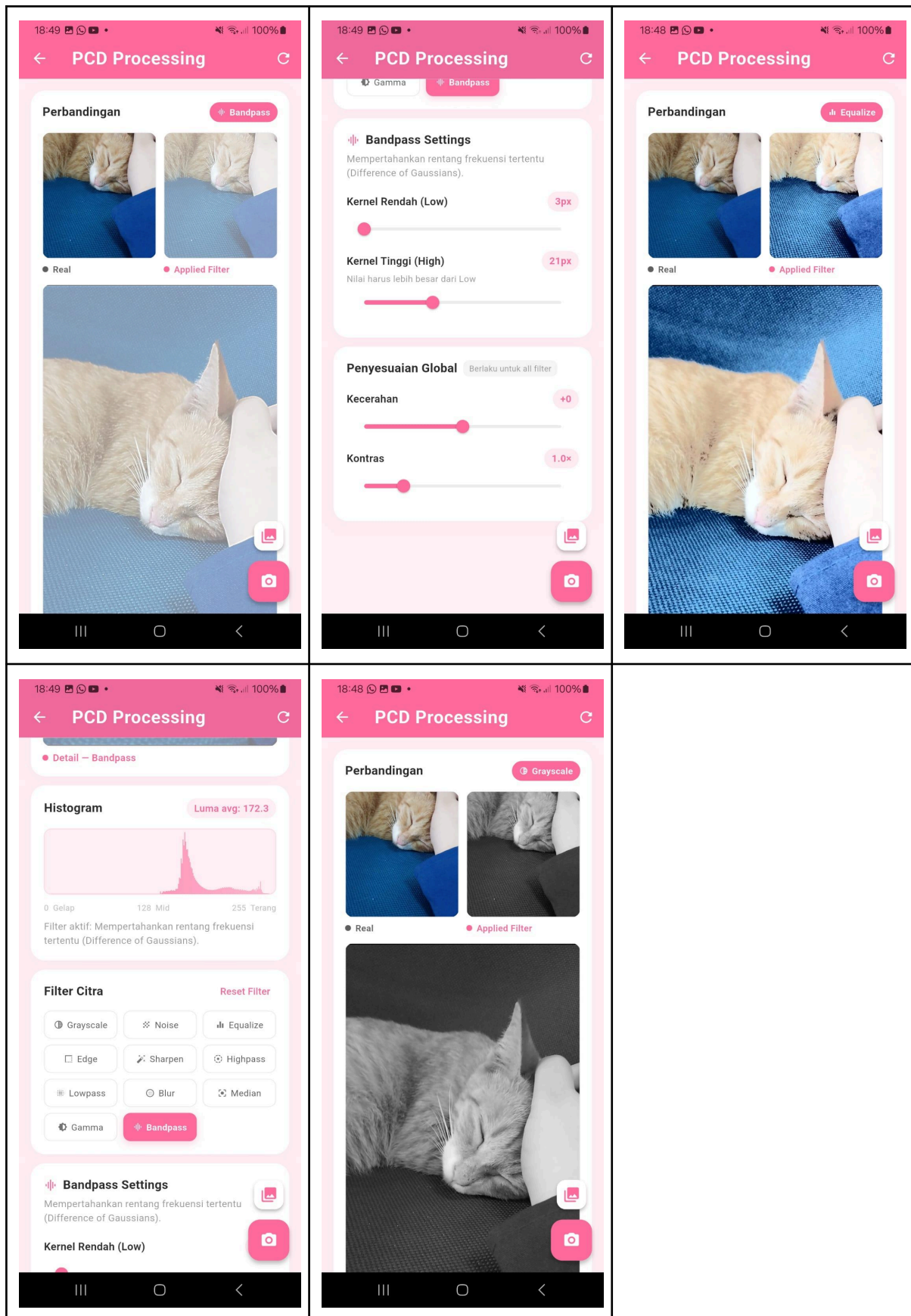
Setelah selesai, tukarkan HP kalian dengan teman sebangku. Cobalah aplikasi mereka dan berikan penilaian sejujur mungkin. Ingat, tujuannya bukan menjatuhkan, tapi saling memperbaiki!

Nama Penilai: Maulana Ishak **Nama Pemilik Aplikasi:** Naira Tahira

No	Kriteria Pengalaman Pengguna (UX)	Skor (1-5)	Catatan "Hal Keren/Saran"
1	Kestabilan Kamera: Preview kamera tidak pecah/lag dan tidak terjadi <i>crash</i> saat berpindah-pindah halaman.	5	
2	Akurasi Overlay: Kotak deteksi (Painter) berada tepat di posisi yang diharapkan dan responsif terhadap perubahan layar.	5	
3	Ergonomi UI: Letak label informasi tidak menghalangi objek utama di kamera dan teks mudah dibaca.	5	
4	Efisiensi Memori: Perangkat tidak mengalami panas berlebih atau perlambatan signifikan saat fitur vision aktif.	5	

Print Screen Aplikasi Teman Anda:





Lesson Learnt (Refleksi Akhir)

Jangan biarkan ilmu hari ini menguap begitu saja. Tuliskan 3 poin utama yang baru kalian sadari atau pahami hari ini.

1. Konsep Baru: Baru tahu kalau kamera di Flutter itu pakai koordinat normalisasi (0.0–1.0) buat bounding box, bukan piksel langsung. Jadi satu set koordinat bisa jalan di layar HP apapun tanpa perlu hitung ulang manual.
2. Kemenangan Kecil: Berhasil fix tampilan kamera yang tadinya stretch/gepeng pakai `AspectRatio` widget. Ternyata kuncinya cuma bungkus CameraPreview di dalam `AspectRatio(aspectRatio: 1 / cameraAspectRatio)` dan kasih background hitam di sisanya.
3. Target Berikutnya: Ingin coba integrasikan model YOLO beneran ke pipeline kamera biar deteksinya real, bukan mock detection pakai timer random.

Penjelasan cara implementasi image processing:

- Grayscale: Merubah RGB menjadi matriks hitam-putih memakai fungsi linear OpenCV `cv.cvtColor(mat, cv.COLOR_BGR2GRAY)`.
- Aritmetika (Tambah, Kurang, Kali): Operasi array memory secara langsung pada pointer memori `mat.data` dalam loop 1 Dimensi. Menghindari overflow warna via fungsi pembatas `.clamp(0, 255)`.
- Inversi: Pembalikan komplementer nilai warna (negatif) yang dioptimasi pada level bits dengan pemanggilan native `cv.bitwiseNOT`.
- Histogram Data: Pembacaan frekuensi sebaran intensitas piksel 0-255 memakai kalkulator probabilitas bawaan `cv.calcHist()` untuk diplot ke UI.
- Histogram Equalization: Merubah citra BGR ke ruang YCrCb lalu memisahkan channel luminance (Y). Meratakan sebaran bayangan pada matrix Y menggunakan `cv.equalizeHist()` tanpa merusak struktur hue pewarnaan original, lalu digabung kembali.
- Histogram Matching: Algoritma manual yang menghitung CDF (Cumulative Distribution Function) dari gambar sumber dan gambar referensi untuk melakukan mapping/matching palet antar piksel.
- Noise Generation:
 - Salt & Pepper / Periodic: Mengandalkan Dart `Random()` generator atau rumus modulo dimensi/grid untuk merusak nilai array matriks pada index tertentu agar menjadi ekstrim (0 atau 255).
 - Gaussian: Memakai generator kurva distribusi spasial normal pada matrix independen dengan `cv.randn()`, yang lalu disisipkan dengan matriks asli via `cv.add`.
- Spatial Filters: Menggunakan jendela konvolusi kernel yang dieksekusi secara instan dengan hardware acceleration via native bindings: `cv.blur` (Mean), `cv.medianBlur` (Median), `cv.gaussianBlur`, serta diferensial turunan tepi buatan `cv.laplacian` (High-pass).
- Fourier Transform (Domain Frekuensi): Mengekstrak sinyal gambar ke mode gelombang dengan mentransformasikan matriks float via `cv.dft(flags: COMPLEX_OUTPUT)`. Matriks bilangan kompleks ini (real & imajiner) dipisahkan nilainya, lalu dirubah ke bentuk polar logaritmik melalui fungsi `cv.magnitude()` dan

cv.log() disusul dengan quadrant shift (pergeseran diagonal) agar low-frequency berada di pusaran matrix spektrum

Hal-hal paling mudah:

Aplikasi Spatial Filter & Konversi: Pemanggilan fungsi filter standar (Gaussian, Blur, Invert, Grayscale, dsb) sangatlah mudah dan singkat karena tinggal memanggil fungsi dari opencv_dart .

Hal-hal paling sulit:

Mengimplementasikan Fourier Transform.

Lampiran lainnya (screenshot-screenshot): (cek [Lampiran](#))

Log LLM: The Fact Check & Twist (Homework)

Menggunakan AI itu boleh, yang tidak boleh adalah menjadi "Zombi AI" (Copy-Paste tanpa mikir). Gunakan template ini setiap kali kalian meminta bantuan ChatGPT/Gemini/Claude.

Contoh Cara Bertanya yang Benar (Prompting)

✗ Salah: *"Buatin kode list di Flutter."* (Terlalu umum, kalian akan bingung bacanya).
✓ Benar: *"Saya punya class CounterController di Dart. Saya ingin menambah List<String> untuk simpan riwayat tiap tombol diklik. Tolong berikan logika murni Dart-nya saja tanpa UI."*

Komponen	Isian Mahasiswa
Pertanyaan (Prompt)	<i>"Gimana cara nambahin data ke List tapi dibatasi cuma 5 data terbaru aja di Dart?"</i>
Jawaban AI (Intisari)	AI menyarankan pakai fungsi insert(0, data) lalu pakai removeLast() kalau panjang list > 5.
The Fact Check	Saya coba di DartPad, ternyata removeLast() bakal error kalau List-nya masih kosong. Jadi saya tambahkan pengecekan if (list.isNotEmpty).
The Twist (Modifikasi)	Saya nggak cuma simpan teks biasa, tapi saya tambahkan jam otomatis pakai DateTime.now() supaya lebih informatif seperti aplikasi logbook asli.

Log LLM Saya:

Komponen	Isian Mahasiswa
Pertanyaan (Prompt)	<i>"Buatkan fitur Smart-Patrol Vision dengan kamera, dengan mock detection, dengan tampilan overlay bounding box, dan control torch/overlay. Pastikan ada auto-dispose resource."</i>
Jawaban AI (Intisari)	AI membuatkan VisionController dengan Timer.periodic untuk simulasi deteksi random tiap 3 detik, DamagePainter untuk gambar bounding box berwarna sesuai severity (D40=merah, D00=kuning), dan WidgetsBindingObserver untuk otomatis lepas kamera saat app ke background.
The Fact Check	Saya coba di HP, ternyata kamera masih ke-stretch secara vertikal karena pakai OverflowBox dengan StackFit.expand. Jadi saya minta perbaikan pakai AspectRatio widget supaya rasio 4:3 kamera dipertahankan, bukan di-stretch full screen.
The Twist (Modifikasi)	Saya tidak hanya pakai kamera buat deteksi, tapi ditambahkan tombol capture yang mengarah ke halaman Image Processing lengkap (grayscale, aritmetika, histogram, equalization, noise, filter, fourier). Jadi satu flow dari kamera langsung ke pengolahan citra.

Komponen	Isian Mahasiswa
Pertanyaan (Prompt)	"Buatkan halaman pengolahan citra (mulai dari grayscale, operasi aritmetika, histogram matching, noise generator, spatial filter, hingga fourier transform) dengan UI tab interaktif. Wajib langsung menggunakan library OpenCV untuk Flutter agar aplikasinya tidak terlalu kompleks."
Jawaban AI (Intisari)	AI membuatkan class ImageProcessingUtils yang mengandalkan fungsi bawaan OpenCV (cv.calcHist, cv.equalizeHist, cv.blur, cv.dft) melalui akses pointer memori Uint8List. Arsitektur komunikasi data dibuat sangat efisien berkat transisi decoding memori ini, sehingga peralihan tiap filter di UI berjalan tanpa adanya lag sama sekali.
The Fact Check	Pada awalnya penerapan Fourier Transform menghasilkan titik magnitudo yang peletakan kuadrannya tidak sesuai. Setelah dievaluasi, hal ini ternyata disebabkan tidak diimplementasikannya getOptimalDFTSize dan algoritma geser kuadran (quadrant shift) untuk gambar hasil Fourier. AI kemudian melengkapi padding matriks kompleks tersebut sehingga frekuensi pusat tepat termanifestasi di tengah.
The Twist (Modifikasi)	Saya menyortir dan hanya menggunakan efek-efek grafis. Fitur-fitur generik yang kurang relevan sengaja saya buang penggunaannya, sehingga interface menjadi jauh lebih ringkas, responsif, dan hanya mencerminkan pengolahan inti di dunia nyata saja.

Rubrik Penilaian Dosen Manajer

Tabel 5.2: Rubrik Penilaian Praktikum Modul 5

Komponen Penilaian	Skor 1 (Kurang)	Skor 3 (Cukup)	Skor 5 (Sangat Baik)	Bobot
Task 2: Vision & Lifecycle Safety	Kamera tidak inisialisasi, <i>crash</i> , atau tidak meminta izin akses (<i>permission</i>).	Kamera berhasil aktif, namun aspek rasio terdistorsi atau terjadi <i>memory leak</i> saat aplikasi di- <i>minimize</i> .	Resource Managed: Kamera stabil, aspek rasio sempurna, dan otomatis <i>dispose/resume</i> sesuai status <i>lifecycle</i> .	25%
Task 3: Canvas & Overlay Logic	Lapisan CustomPaint tidak muncul atau menutupi preview kamera secara permanen.	<i>Overlay</i> muncul namun posisi statis (tidak responsif) atau teks label sulit dibaca (kontras rendah).	Precise Rendering: Indikator visual rapi, menggunakan TextPainter yang informatif, dan posisi tetap presisi di berbagai ukuran layar.	25%
Task 4: Mock Detection & Scaling	Tidak ada simulasi deteksi objek atau koordinat kotak deteksi meleset jauh dari target.	Simulasi deteksi jalan rusak berjalan, namun terjadi <i>flicker</i> atau kotak tidak proporsional terhadap layar.	Rigid Scaling: Logika transformasi koordinat akurat; kotak deteksi bergerak dinamis dan proporsional terhadap <i>Logical Pixels</i> .	25%
Homework: UX & Portofolio	Tidak ada fitur kontrol tambahan (Flash/Toggle) atau UI terlihat tidak profesional.	Fitur kontrol ada namun sering <i>buggy</i> atau tidak sinkron dengan status asli <i>hardware</i> kamera.	Pro Grade UX: Kontrol <i>Flash/Torch</i> sangat responsif, <i>Empty State</i> informatif, dan estetika visual konsisten dengan tema aplikasi.	25%



Rubrik Penilaian Log LLM (Integritas)

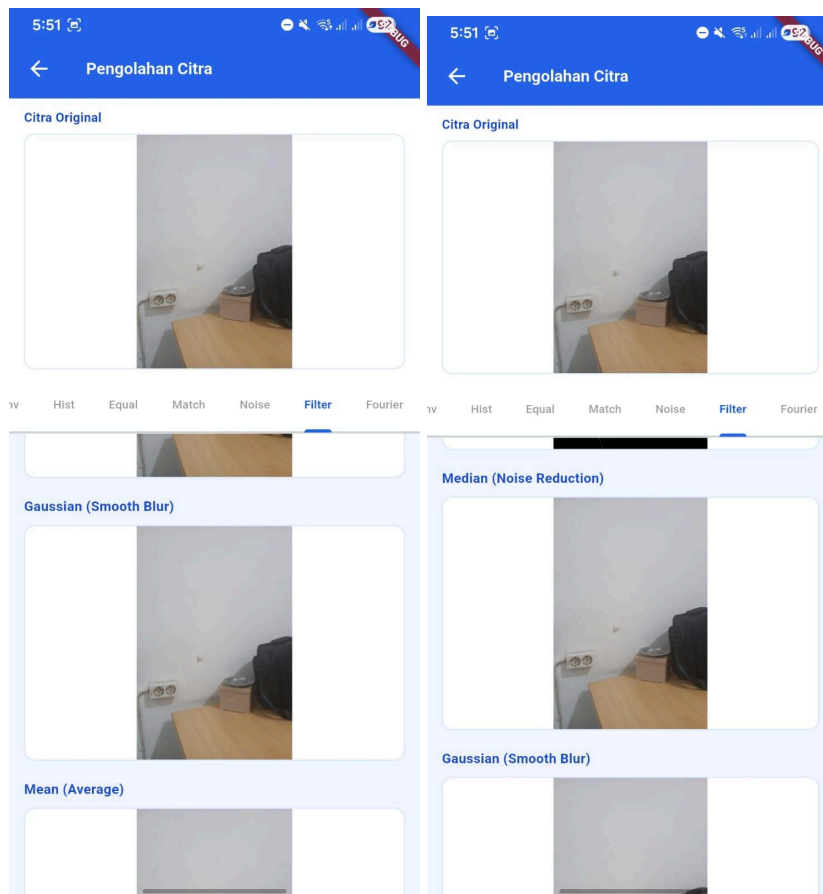
Tabel 2.3: Rubrik Penilaian Integritas Log LLM.

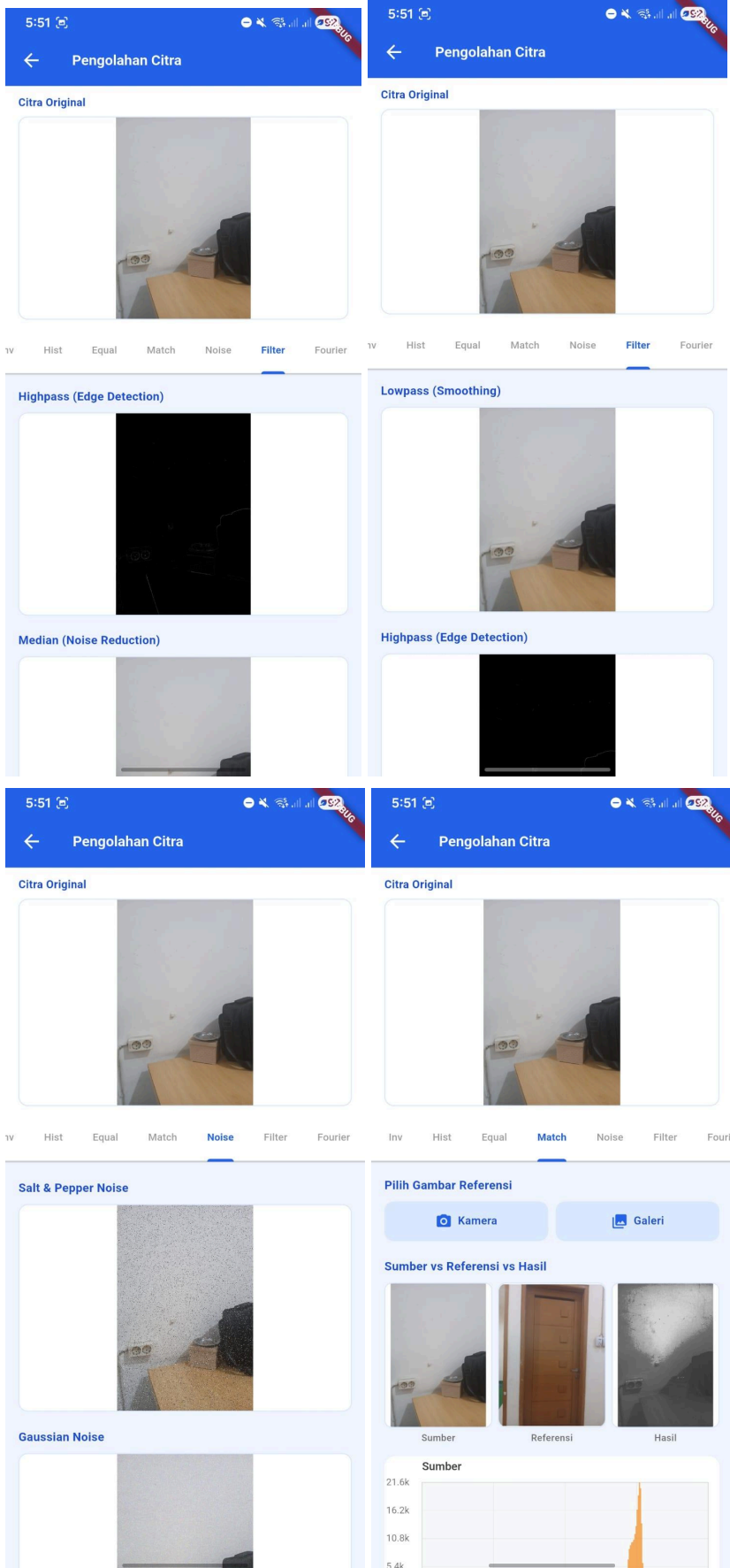
Kriteria	Skor 1	Skor 3	Skor 5
Kejujuran	Mengaku tidak pakai AI padahal	Mencantumkan prompt tapi jawaban AI disalin bulat-bulat.	Terbuka soal bantuan AI dan mencantumkan prompt dengan jelas.

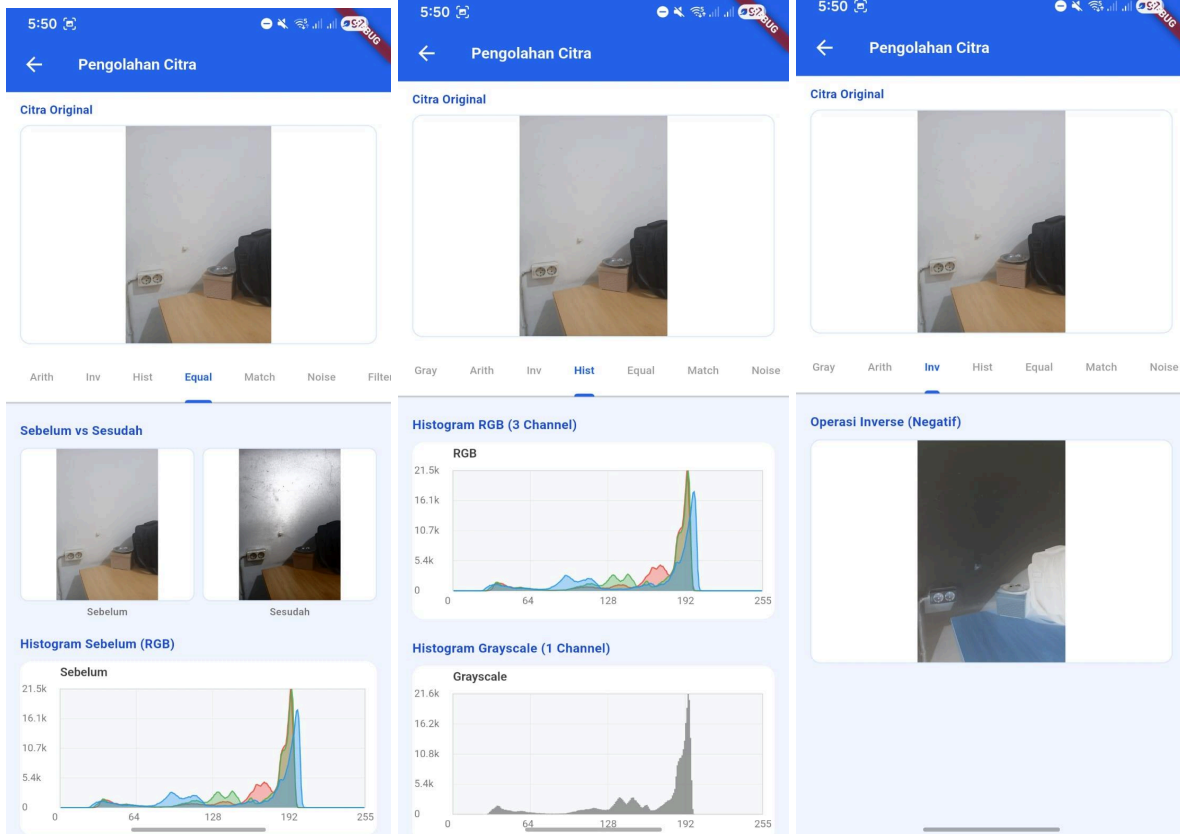
	kode sangat kompleks.		
Analisa (Fact Check)	Tidak ada pengecekan ulang.	Mengecek apakah kode jalan atau tidak saja.	Mampu menemukan celah atau kelemahan dari saran AI.
Kreativitas (Twist)	Kode 100% sama dengan saran AI.	Mengubah nama variabel saja.	Mengintegrasikan saran AI ke dalam struktur SRP yang sudah dibuat.

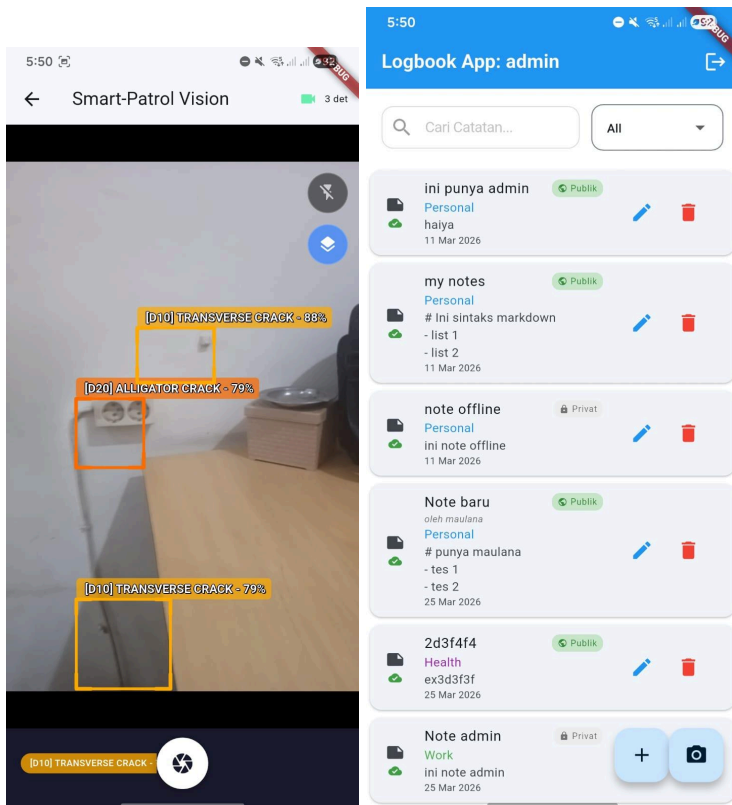
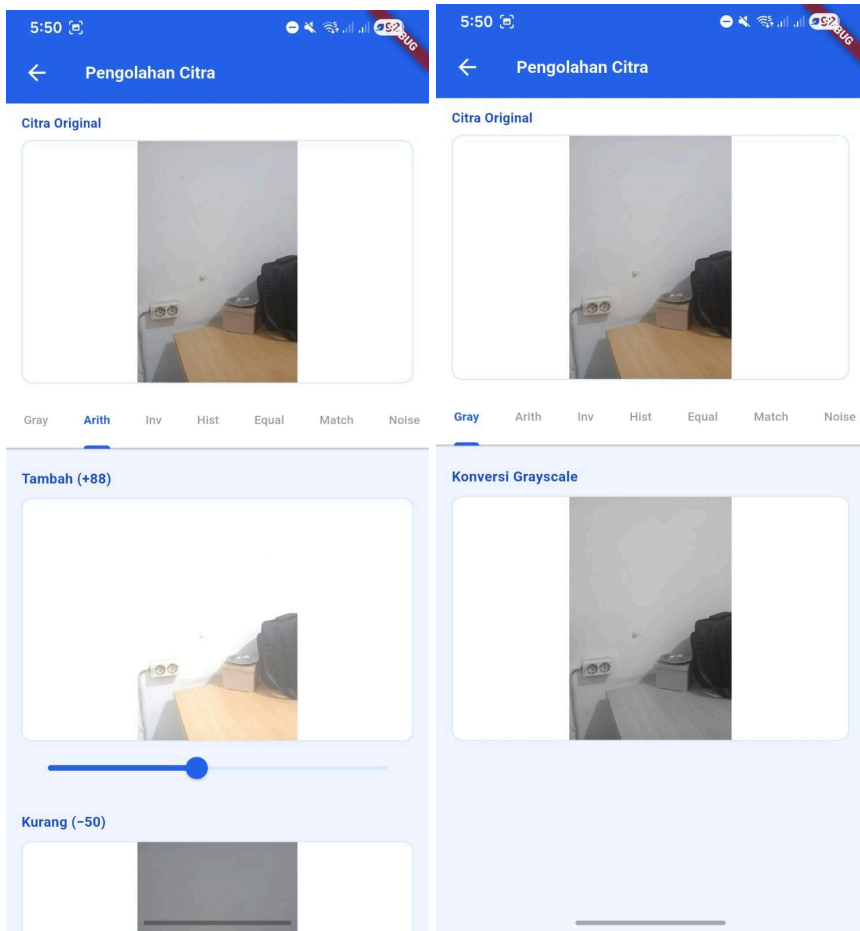
Lampiran

Screenshot UI Aplikasi:

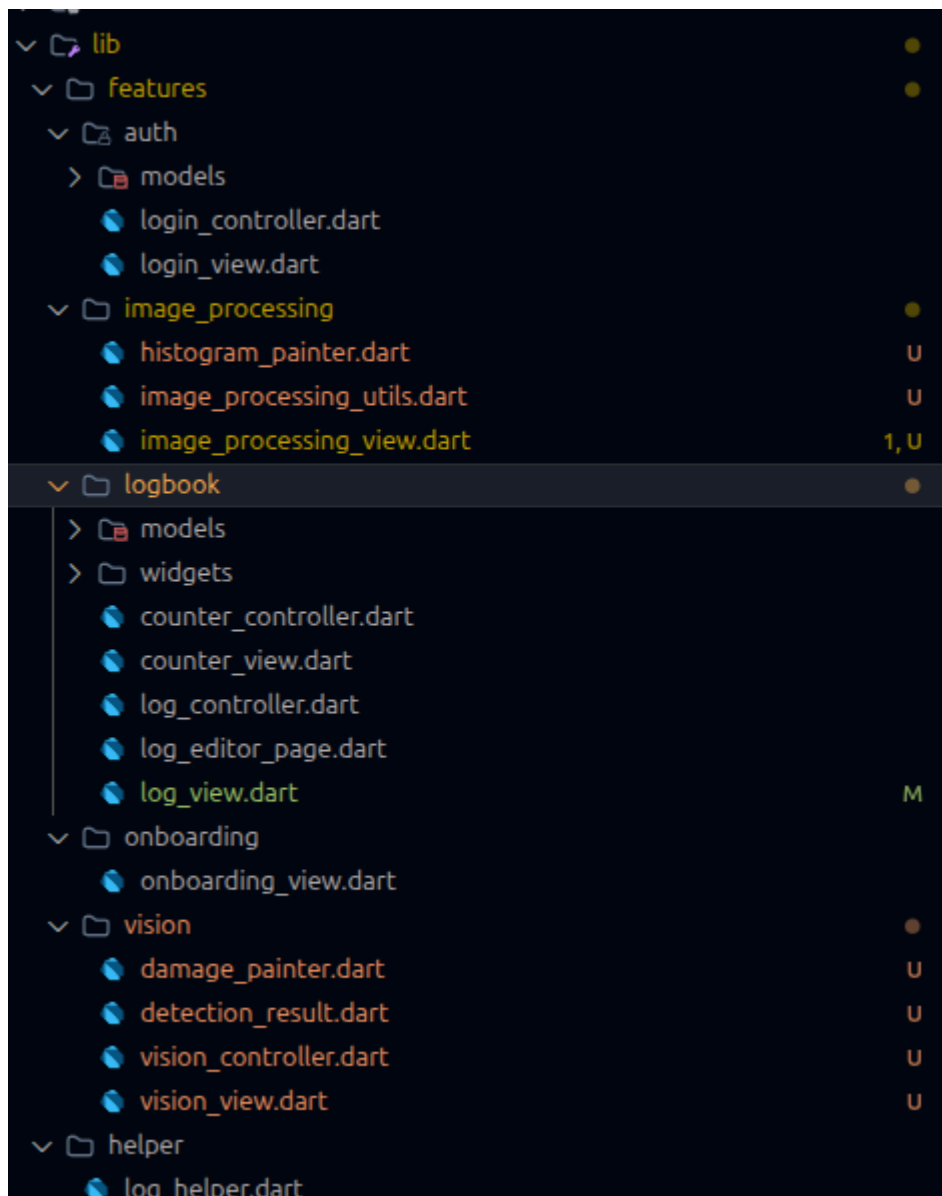








Screenshot struktur folder code:



Kode yang menerapkan PCD:

```
import 'dart:math' show Random;

import 'dart:typed_data';

import 'package:opencv_dart/opencv_dart.dart' as cv;

/// Utility class untuk pengolahan citra menggunakan OpenCV (native
performance).

/// Semua input dan output adalah Uint8List (representasi byte JPG).

class ImageProcessingUtils {
```



```

    static cv.Mat _decode(UInt8List bytes) => cv.imdecode(bytes,
cv.IMREAD_COLOR);

    static cv.Mat _decodeGray(UInt8List bytes) =>

        cv.imdecode(bytes, cv.IMREAD_GRAYSCALE);

    static UInt8List _encode(cv.Mat mat) => cv.imencode('.jpg', mat).$2;

    // ===== 1. GRAYSCALE =====

    static UInt8List toGrayscale(UInt8List sourceBytes) {

        final mat = _decode(sourceBytes);

        final gray = cv.cvtColor(mat, cv.COLOR_BGR2GRAY);

        return _encode(gray);

    }

    // ===== 2. ARITMETIKA =====

    // Menggunakan manipulasi memori langsung untuk aritmetika sederhana
    agar aman tanpa ribet tipe scalar OpenCV

    static UInt8List arithmeticAdd(UInt8List sourceBytes, int value) {

        final mat = _decode(sourceBytes);

        final data = mat.data;

        for (int i = 0; i < data.length; i++) {

            data[i] = (data[i] + value).clamp(0, 255);

        }

        return _encode(mat);

    }

    static UInt8List arithmeticSubtract(UInt8List sourceBytes, int value)
{

        final mat = _decode(sourceBytes);

```

```

    final data = mat.data;

    for (int i = 0; i < data.length; i++) {
        data[i] = (data[i] - value).clamp(0, 255);
    }

    return _encode(mat);
}

static Uint8List arithmeticMultiply(Uint8List sourceBytes, double
factor) {
    final mat = _decode(sourceBytes);
    final data = mat.data;
    for (int i = 0; i < data.length; i++) {
        data[i] = (data[i] * factor).round().clamp(0, 255);
    }
    return _encode(mat);
}

// ===== 3. INVERSE =====

static Uint8List inverse(Uint8List sourceBytes) {
    final mat = _decode(sourceBytes);
    final inv = cv.bitwiseNOT(mat);
    return _encode(inv);
}

// ===== 4. HISTOGRAM =====

static List<int> computeGrayscaleHistogram(Uint8List sourceBytes) {
    final gray = _decodeGray(sourceBytes);

```

```

final vecMat = cv.VecMat.fromList([gray]);

final channels = cv.VecI32.fromList([0]);

final histSize = cv.VecI32.fromList([256]);

final ranges = cv.VecF32.fromList([0.0, 256.0]);


final hist = cv.calcHist(
    vecMat,
    channels,
    cv.Mat.empty(),
    histSize,
    ranges,
);

final f32Data = hist.data.buffer.asFloat32List();

return f32Data.map((e) => e.round()).toList();
}

static List<List<int>> computeRgbHistogram(Uint8List sourceBytes) {
    final mat = _decode(sourceBytes);

    final vecMat = cv.VecMat.fromList([mat]);

    final histSize = cv.VecI32.fromList([256]);

    final ranges = cv.VecF32.fromList([0.0, 256.0]);

    // OpenCV BGR order

    final bHist = cv.calcHist(
        vecMat,
        cv.VecI32.fromList([0]),
        cv.Mat.empty(),
        histSize,
        ranges,
    );
}

```

```

);

final gHist = cv.calcHist(
    vecMat,
    cv.VecI32.fromList([1]),
    cv.Mat.empty(),
    histSize,
    ranges,
);

final rHist = cv.calcHist(
    vecMat,
    cv.VecI32.fromList([2]),
    cv.Mat.empty(),
    histSize,
    ranges,
);

return [
    rHist.data.buffer.asFloat32List().map((e) => e.round()).toList(),
    gHist.data.buffer.asFloat32List().map((e) => e.round()).toList(),
    bHist.data.buffer.asFloat32List().map((e) => e.round()).toList(),
];
}

// ===== 5. HISTOGRAM EQUALIZATION =====

static Uint8List histogramEqualization(Uint8List sourceBytes) {
    final mat = _decode(sourceBytes);
    // 1. Ubah ke YCrCb
    final ycrb = cv.cvtColor(mat, cv.COLOR_BGR2YCrCb);

```

```

// 2. Split channel (Y, Cr, Cb)

final channels = cv.split(ycrCb);

// 3. Equalize channel Y (index 0)

final yEq = cv.equalizeHist(channels[0]);

channels[0] = yEq;

// 4. Merge kembali

final merged = cv.merge(channels);

// 5. Convert kembali ke BGR

final result = cv.cvtColor(merged, cv.COLOR_YCrCb2BGR);

return _encode(result);
}

// ===== 6. HISTOGRAM MATCHING =====

static UInt8List histogramMatchingFromReference(
    UInt8List sourceBytes,
    UInt8List refBytes,
) {
    // Kita gunakan teknik mapping manual karena OpenCV C++ tidak punya
    // fungsi `matchHist` standar

    final srcGray = _decodeGray(sourceBytes);
    final refGray = _decodeGray(refBytes);

    final srcHistF = cv
        .calcHist(
            cv.VecMat.fromList([srcGray]),
            cv.VecI32.fromList([0]),
            cv.Mat.empty(),
            cv.VecI32.fromList([256]),

```

```

        cv.VecF32.fromList([0.0, 256.0]),
    )

    .data

    .buffer

    .asFloat32List();

final refHistF = cv

    .calcHist(

        cv.VecMat.fromList([refGray]),

        cv.VecI32.fromList([0]),

        cv.Mat.empty(),

        cv.VecI32.fromList([256]),

        cv.VecF32.fromList([0.0, 256.0]),

    )

    .data

    .buffer

    .asFloat32List();


int srcTotal = srcGray.rows * srcGray.cols;
int refTotal = refGray.rows * refGray.cols;

final srcCdf = List<double>.filled(256, 0);
final refCdf = List<double>.filled(256, 0);

srcCdf[0] = srcHistF[0] / srcTotal;
refCdf[0] = refHistF[0] / refTotal;
for (int i = 1; i < 256; i++) {

    srcCdf[i] = srcCdf[i - 1] + srcHistF[i] / srcTotal;

    refCdf[i] = refCdf[i - 1] + refHistF[i] / refTotal;

}

```

```

final mapping = List<int>.filled(256, 0);

for (int r = 0; r < 256; r++) {

    int bestZ = 0;

    double bestDiff = double.infinity;

    for (int z = 0; z < 256; z++) {

        final double diff = (refCdf[z] - srcCdf[r]).abs();

        if (diff < bestDiff) {

            bestDiff = diff;

            bestZ = z;

        }

    }

    mapping[r] = bestZ;

}

// Apply mapping

final srcData = srcGray.data;

for (int i = 0; i < srcData.length; i++) {

    srcData[i] = mapping[srcData[i]];

}

return _encode(srcGray);

}

// ===== 7. NOISE =====

static Uint8List noiseSaltAndPepper(

    Uint8List sourceBytes, {

        double amount = 0.05,

```

```

    }) {

        final mat = _decode(sourceBytes);

        final data = mat.data;

        final int total = (data.length / 3 * amount).round();

        final rng = Random(); // from dart:math

        for (int i = 0; i < total; i++) {

            final int row = rng.nextInt(mat.rows);

            final int col = rng.nextInt(mat.cols);

            final int val = rng.nextInt(2) == 0 ? 0 : 255;

            final int idx = (row * mat.cols + col) * 3;

            data[idx] = val;

            data[idx + 1] = val;

            data[idx + 2] = val;

        }

        return _encode(mat);
    }

    static Uint8List noiseGaussian(Uint8List sourceBytes, {double sigma =
25}) {

        final mat = _decode(sourceBytes);

        // Buat noise dengan mean 0 dan standard deviation = sigma

        final noise = cv.Mat.zeros(mat.rows, mat.cols, mat.type);

        cv.randn(noise, cv.Scalar(0, 0, 0, 0), cv.Scalar(sigma, sigma,
sigma, 0));

        final noisy = cv.add(mat, noise);

        return _encode(noisy);
    }

    static Uint8List noisePeriodic(

        Uint8List sourceBytes, {

```



```

    int stripeWidth = 8,

    int spacing = 40,
)) {

    final mat = _decode(sourceBytes);

    final data = mat.data;

    final int period = stripeWidth + spacing;

    final int cols = mat.cols;

    for (int y = 0; y < mat.rows; y++) {

        for (int x = 0; x < cols; x++) {

            if ((x + y) % period < stripeWidth) {

                final int idx = (y * cols + x) * 3;

                data[idx] = (data[idx] * 0.2).round().clamp(0, 255);

                data[idx + 1] = (data[idx + 1] * 0.2).round().clamp(0, 255);

                data[idx + 2] = (data[idx + 2] * 0.2).round().clamp(0, 255);

            }

        }

    }

    return _encode(mat);

}

// ===== 8. FILTER =====

static Uint8List filterLowpass(Uint8List sourceBytes) {

    final mat = _decode(sourceBytes);

    // Lowpass = Blur standard

    final blur = cv.blur(mat, (3, 3));

    return _encode(blur);

}

```

```

static Uint8List filterHighpass(Uint8List sourceBytes) {
    final mat = _decode(sourceBytes);
    // Laplacian edge detection, depth is CV_8U (which is 0)
    final laplacian = cv.laplacian(mat, 0);
    return _encode(laplacian);
}

static Uint8List filterMedian(Uint8List sourceBytes) {
    final mat = _decode(sourceBytes);
    final median = cv.medianBlur(mat, 3);
    return _encode(median);
}

static Uint8List filterGaussian(Uint8List sourceBytes) {
    final mat = _decode(sourceBytes);
    final gaussian = cv.gaussianBlur(mat, (5, 5), 1.0);
    return _encode(gaussian);
}

static Uint8List filterMean(Uint8List sourceBytes) {
    return filterLowpass(sourceBytes);
}

// ===== 9. FOURIER =====

static Uint8List fourierMagnitudeSpectrum(Uint8List sourceBytes) {
    final mat = _decode(sourceBytes);
    final gray = cv.cvtColor(mat, cv.COLOR_BGR2GRAY);

```

```

// get optimal dft size

int m = cv.getOptimalDFTSize(gray.rows);

int n = cv.getOptimalDFTSize(gray.cols);

final padded = cv.copyMakeBorder(

    gray,

    0,

    m - gray.rows,

    0,

    n - gray.cols,

    cv.BORDER_CONSTANT,

);

// convert to float

final floatMat = padded.convertTo(cv.MatType.CV_32FC1);

// perform dft

final dft = cv.dft(floatMat, flags: cv.DFT_COMPLEX_OUTPUT);

// split and magnitude

final planes = cv.split(dft);

final mag = cv.magnitude(planes[0], planes[1]);

// mag += 1

final ones = cv.Mat.ones(mag.rows, mag.cols, cv.MatType.CV_32FC1);

var logMag = cv.add(mag, ones);

cv.log(logMag, dst: logMag);

// shift quadrants

```

```

int cx = logMag.cols ~/ 2;

int cy = logMag.rows ~/ 2;

final q0 = logMag.region(cv.Rect(0, 0, cx, cy));

final q1 = logMag.region(cv.Rect(cx, 0, cx, cy));

final q2 = logMag.region(cv.Rect(0, cy, cx, cy));

final q3 = logMag.region(cv.Rect(cx, cy, cx, cy));


final tmp = cv.Mat.empty();

q0.copyTo(tmp);

q3.copyTo(q0);

tmp.copyTo(q3);


q1.copyTo(tmp);

q2.copyTo(q1);

tmp.copyTo(q2);


// normalize 0-255

final norm = cv.normalize(

    logMag,

    logMag,

    alpha: 0,

    beta: 255,

    normType: cv.NORM_MINMAX,

);

final result = norm.convertTo(cv.MatType.CV_8UC1);

return _encode(result);
}
}

```